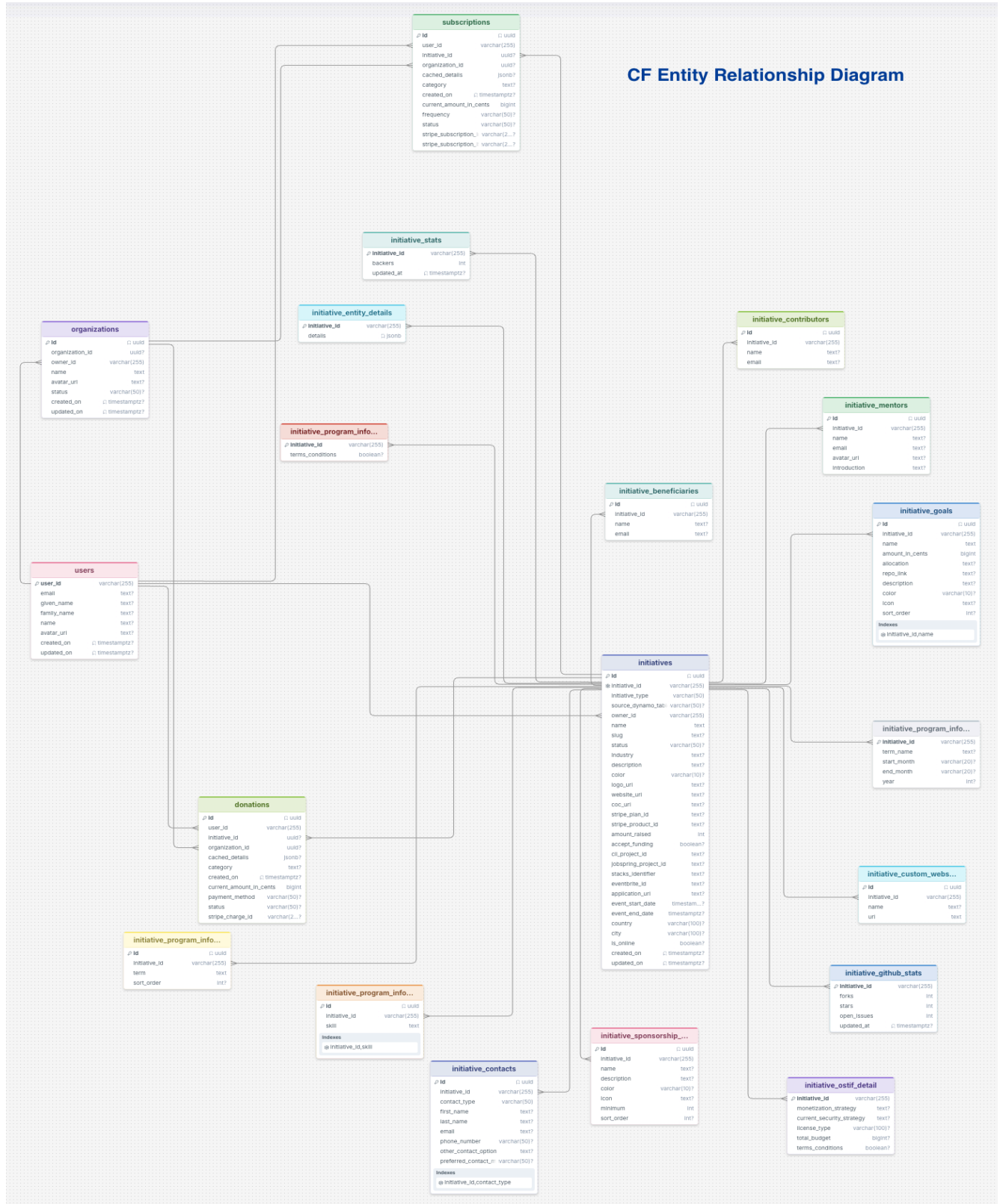


CF Entity Relationship Diagram



Normalized Schema Definition

SQL

```
-- =====
-- Migration: Normalised Initiatives Schema
-- Version: 2.0.0
-- Created: 2026-05-07
-- Description: Fully normalised schema.
-- initiatives merges lff-prod-projects + lff-prod-entities.
--
-- Excluded (no DynamoDB write path / computed at read-time):
-- balance          - computed from ledger at read time
-- funding_status   - computed from ledger + subscription summaries
-- entity_stats     - EntityStats.Sponsors computed at read time
-- sponsors          - GetEntitySponsors pushes to ES only; SaveEntity never
--                   called with sponsors populated
-- diversity        - GetDiversity fetches external API at read time
-- vulnerability    - GetVulnerabilitySummary fetches external service
-- badges           - convertProjectToDynamoRepresentation never assigns Badges
-- total_raised     - ProjectStats.TotalRaised has no update path (always 0)
-- =====
```

BEGIN;

```
-- =====
-- TABLE: users
-- =====
CREATE TABLE IF NOT EXISTS users (
  user_id    VARCHAR(255) PRIMARY KEY,
  email      TEXT,
  given_name TEXT,
  family_name TEXT,
  name       TEXT,
  avatar_url TEXT,
  created_on TIMESTAMPTZ DEFAULT NOW(),
  updated_on TIMESTAMPTZ DEFAULT NOW()
);
-- =====
-- TABLE: organizations
-- =====
CREATE TABLE IF NOT EXISTS organizations (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id   UUID,
  owner_id          VARCHAR(255) NOT NULL REFERENCES users(user_id),
  name              TEXT NOT NULL,
```

```

    avatar_url      TEXT,
    status          VARCHAR(50),
    created_on      TIMESTAMPTZ DEFAULT NOW(),
    updated_on      TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- TABLE: initiatives (merged projects + entities)
--
-- initiative_id is VARCHAR(255) - DynamoDB stores projectId / entityId
--   as UUID strings but the type must not be narrowed to UUID because
--   the column is a natural key shared across both source tables.
--
-- source_dynamo_table is "migration only" and will be dropped in Phase 5
--   after the application no longer writes to DynamoDB.
-- =====
CREATE TABLE IF NOT EXISTS initiatives (
    -- surrogate PK (referenced by donations + subscriptions FK)
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),

    -- natural identity
    initiative_id      VARCHAR(255) NOT NULL UNIQUE,
    initiative_type    VARCHAR(50)  NOT NULL,
    source_dynamo_table VARCHAR(50),    -- 'projects' | 'entities' - drop
post-cutover

    -- ownership
    owner_id           VARCHAR(255) NOT NULL REFERENCES users(user_id),

    -- core display fields
    name               TEXT           NOT NULL,
    slug               TEXT,
    status             VARCHAR(50),
    industry           TEXT,
    description        TEXT,
    color              VARCHAR(10),
    logo_url           TEXT,
    website_url        TEXT,
    coc_url            TEXT,

    -- financial / platform IDs
    stripe_plan_id     TEXT,
    stripe_product_id  TEXT,
    amount_raised      INTEGER       NOT NULL DEFAULT 0,
    accept_funding     BOOLEAN       DEFAULT false,

```

```

-- project-only fields (SP write path)
cii_project_id      TEXT,
jobspring_project_id TEXT,                                -- written only when
programInfo.budget.amountInCents > 0
stacks_identifier   TEXT,

-- entity-only fields (SE / SE-reflect write path)
eventbrite_id       TEXT,
application_url      TEXT,
event_start_date     TIMESTAMPTZ,
event_end_date       TIMESTAMPTZ,
country              VARCHAR(100),
city                 VARCHAR(100),
is_online            BOOLEAN      DEFAULT false,

created_on           TIMESTAMPTZ  DEFAULT NOW(),
updated_on           TIMESTAMPTZ  DEFAULT NOW()
);

-- =====
-- TABLE: initiative_goals
-- Projects : projectDetails.{development,marketing,meetups,travel,
--            bugBounty,documentation,programInfo,other} - each carries
--            Budget{amount, allocation}; development also carries repoLink.
-- Entities : entity.Goals[] - each Goal{name,description,goalColor,
--            goalIcon, budget{amount,allocation}}
-- =====
CREATE TABLE IF NOT EXISTS initiative_goals (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  initiative_id VARCHAR(255) NOT NULL REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
  name        TEXT          NOT NULL,
  amount_in_cents BIGINT     NOT NULL DEFAULT 0,
  allocation   TEXT,
  repo_link    TEXT,        -- project 'development' category only
  description  TEXT,        -- entity goals only
  color        VARCHAR(10),  -- entity goals only (goalColor)
  icon         TEXT,        -- entity goals only (goalIcon)
  sort_order   INTEGER      DEFAULT 0,
  UNIQUE (initiative_id, name)
);

```

```

-- =====
-- TABLE: initiative_beneficiaries
-- Projects : projectDetails.beneficiaries[]
-- Entities : entity.Beneficiary[] (json key: "beneficiaries")
--           Also updated by UpdateEntityBeneficiaries (sets entity.Beneficiary
--           then calls SaveEntity).
-- =====
CREATE TABLE IF NOT EXISTS initiative_beneficiaries (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  initiative_id VARCHAR(255) NOT NULL REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
  name        TEXT,
  email       TEXT
);

-- =====
-- TABLE: initiative_custom_websites
-- Projects : projectDetails.customWebsites[]
-- Entities : SE-reflect via EditEntityRequest.CustomWebsites
-- =====
CREATE TABLE IF NOT EXISTS initiative_custom_websites (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  initiative_id VARCHAR(255) NOT NULL REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
  name        TEXT,
  url         TEXT          NOT NULL
);

-- =====
-- TABLE: initiative_contributors (project only)
-- Projects : projectDetails.Contributors
--           convertProjectToDynamoRepresentation line ~927
-- =====
CREATE TABLE IF NOT EXISTS initiative_contributors (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  initiative_id VARCHAR(255) NOT NULL REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
  name        TEXT,
  email       TEXT
);

```

```

-- =====
-- TABLE: initiative_mentors (project programInfo only)
-- Projects : projectDetails.programInfo.mentor[]
--           Each Mentor{name, email, avatarURL, introduction}
-- =====
CREATE TABLE IF NOT EXISTS initiative_mentors (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  initiative_id VARCHAR(255) NOT NULL REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
  name        TEXT,
  email       TEXT,
  avatar_url  TEXT,
  introduction TEXT
);

-- =====
-- TABLE: initiative_program_info_terms (project programInfo only)
-- Projects : projectDetails.programInfo.terms[] ([]string)
-- =====
CREATE TABLE IF NOT EXISTS initiative_program_info_terms (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  initiative_id VARCHAR(255) NOT NULL REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
  term        TEXT          NOT NULL,
  sort_order  INTEGER        DEFAULT 0
);

-- =====
-- TABLE: initiative_program_info_skills (project programInfo only)
-- Projects : projectDetails.programInfo.skills[] ([]string)
-- =====
CREATE TABLE IF NOT EXISTS initiative_program_info_skills (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  initiative_id VARCHAR(255) NOT NULL REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
  skill       TEXT          NOT NULL,
  UNIQUE (initiative_id, skill)
);

-- =====
-- TABLE: initiative_program_info_config (project programInfo only - 0-or-1 per
initiative)
-- Projects : projectDetails.programInfo.termsConditions (bool)
-- =====
CREATE TABLE IF NOT EXISTS initiative_program_info_config (

```

```

    initiative_id    VARCHAR(255) PRIMARY KEY REFERENCES initiatives(initiative_id)
ON DELETE CASCADE,
    terms_conditions BOOLEAN      DEFAULT false
);

-- =====
-- TABLE: initiative_program_info_custom_term (project programInfo only - 0-or-1
per initiative)
-- Projects : projectDetails.programInfo.customTerm
--           CustomTerm{termName, startMonth, endMonth, year}
--           Only present when termName is non-empty.
-- =====
CREATE TABLE IF NOT EXISTS initiative_program_info_custom_term (
    initiative_id VARCHAR(255) PRIMARY KEY REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
    term_name     TEXT,
    start_month   VARCHAR(20),
    end_month     VARCHAR(20),
    year          INTEGER
);

-- =====
-- TABLE: initiative_sponsorship_tiers (entity only)
-- Entities : SE-reflect via EditEntityRequest.SponsorshipTiers
--           SponsorshipTier{name, description, color, icon, minimum}
-- =====
CREATE TABLE IF NOT EXISTS initiative_sponsorship_tiers (
    id            UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    initiative_id VARCHAR(255) NOT NULL REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
    name          TEXT,
    description   TEXT,
    color         VARCHAR(10),
    icon         TEXT,
    minimum       INTEGER       NOT NULL DEFAULT 0,
    sort_order    INTEGER       DEFAULT 0
);

```

```

-- =====
-- TABLE: initiative_ostif_detail (ostif entity type only - 0-or-1 per
initiative)
-- Entities : SE-reflect via EditEntityRequest.Detail (domain.Detail struct).
--             detail.totalBudget is also consumed explicitly:
--             entity.FundingStatus.TotalAnnualGoalInCents =
int64(editEntityRequest.Detail.TotalBudget)
-- =====
CREATE TABLE IF NOT EXISTS initiative_ostif_detail (
    initiative_id          VARCHAR(255) PRIMARY KEY REFERENCES
initiatives(initiative_id) ON DELETE CASCADE,
    monetization_strategy  TEXT,
    current_security_strategy TEXT,
    license_type           VARCHAR(100),
    total_budget           BIGINT      DEFAULT 0,
    terms_conditions       BOOLEAN     DEFAULT false
);

-- =====
-- TABLE: initiative_contacts (ostif entity type only)
-- Entities : entity.Detail.PrimaryContact, .SecondaryContact, .TechnicalLead
--             Contact{firstName, lastName, email, phoneNumber,
--             otherContactOption, preferredContactMethod}
-- =====
CREATE TABLE IF NOT EXISTS initiative_contacts (
    id                    UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    initiative_id         VARCHAR(255) NOT NULL REFERENCES
initiatives(initiative_id) ON DELETE CASCADE,
    contact_type          VARCHAR(50)  NOT NULL, -- 'primary' | 'secondary' |
'technical_lead'
    first_name            TEXT,
    last_name             TEXT,
    email                 TEXT,
    phone_number          VARCHAR(50),
    other_contact_option   TEXT,
    preferred_contact_method VARCHAR(50),
    UNIQUE (initiative_id, contact_type)
);

```

```

-- =====
-- TABLE: initiative_github_stats (project only - 1-to-1 cached)
-- Updated by UpdateGithubDataCache → targeted UpdateItem on
--   cachedDetails.githubStats.{forks, openIssues, stars}
-- Also initialised at project creation via CachedDetails.GithubStats.
-- =====
CREATE TABLE IF NOT EXISTS initiative_github_stats (
  initiative_id VARCHAR(255) PRIMARY KEY REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
  forks          INTEGER      NOT NULL DEFAULT 0,
  stars          INTEGER      NOT NULL DEFAULT 0,
  open_issues    INTEGER      NOT NULL DEFAULT 0,
  updated_at     TIMESTAMPTZ  DEFAULT NOW()
);

-- =====
-- TABLE: initiative_stats (project only - backers count only)
-- total_raised excluded: no active write path (ProjectStats.TotalRaised always
--   0).
-- Updated by UpdateProjectStats → targeted UpdateItem on
--   cachedDetails.projectStats.backers ± 1
-- =====
CREATE TABLE IF NOT EXISTS initiative_stats (
  initiative_id VARCHAR(255) PRIMARY KEY REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
  backers       INTEGER      NOT NULL DEFAULT 0,
  updated_at    TIMESTAMPTZ  DEFAULT NOW()
);

-- =====
-- TABLE: initiative_entity_details (entity only - JSONB)
-- entity.EntityDetails is map[string]string with application-defined keys.
-- Serialised by dynamodbattribute.MarshalMap under the 'entityDetails' key.
-- =====
CREATE TABLE IF NOT EXISTS initiative_entity_details (
  initiative_id VARCHAR(255) PRIMARY KEY REFERENCES initiatives(initiative_id) ON
DELETE CASCADE,
  details       JSONB        NOT NULL DEFAULT '{}'
);

```

```

-- =====
-- TABLE: donations
-- Merged from lff-prod-donations + lff-prod-entity-donations
-- initiative_id references the surrogate UUID PK on initiatives(id)
-- =====
CREATE TABLE IF NOT EXISTS donations (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           VARCHAR(255) NOT NULL REFERENCES users(user_id),
  initiative_id     UUID                REFERENCES initiatives(id),
  organization_id   UUID                REFERENCES organizations(id),
  cached_details    JSONB,
  category          TEXT,
  created_on        TIMESTAMPTZ DEFAULT NOW(),
  current_amount_in_cents BIGINT        NOT NULL,
  payment_method    VARCHAR(50),
  status            VARCHAR(50),
  stripe_charge_id  VARCHAR(255)
);

-- =====
-- TABLE: subscriptions
-- Merged from lff-prod-subscriptions + lff-prod-entity-subscriptions
-- initiative_id references the surrogate UUID PK on initiatives(id)
-- =====
CREATE TABLE IF NOT EXISTS subscriptions (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           VARCHAR(255) NOT NULL REFERENCES users(user_id),
  initiative_id     UUID                REFERENCES initiatives(id),
  organization_id   UUID                REFERENCES organizations(id),
  cached_details    JSONB,
  category          TEXT,
  created_on        TIMESTAMPTZ DEFAULT NOW(),
  current_amount_in_cents BIGINT        NOT NULL,
  frequency         VARCHAR(50),
  status            VARCHAR(50),
  stripe_subscription_id VARCHAR(255),
  stripe_subscription_item_id VARCHAR(255)
);

```

```

-- =====
-- INDEXES
-- =====

-- organizations
CREATE INDEX IF NOT EXISTS idx_organizations_owner_id ON
organizations(owner_id);

-- initiatives (core)
CREATE INDEX IF NOT EXISTS idx_initiatives_initiative_id ON
initiatives(initiative_id);
CREATE INDEX IF NOT EXISTS idx_initiatives_owner_id ON
initiatives(owner_id);
CREATE INDEX IF NOT EXISTS idx_initiatives_slug ON
initiatives(slug);
CREATE INDEX IF NOT EXISTS idx_initiatives_status ON
initiatives(status);
CREATE INDEX IF NOT EXISTS idx_initiatives_type ON
initiatives(initiative_type);
CREATE INDEX IF NOT EXISTS idx_initiatives_amount_raised ON
initiatives(amount_raised DESC);

-- initiative child tables
CREATE INDEX IF NOT EXISTS idx_initiative_goals_iid ON
initiative_goals(initiative_id);
CREATE INDEX IF NOT EXISTS idx_initiative_beneficiaries_iid ON
initiative_beneficiaries(initiative_id);
CREATE INDEX IF NOT EXISTS idx_initiative_custom_websites_iid ON
initiative_custom_websites(initiative_id);
CREATE INDEX IF NOT EXISTS idx_initiative_contributors_iid ON
initiative_contributors(initiative_id);
CREATE INDEX IF NOT EXISTS idx_initiative_mentors_iid ON
initiative_mentors(initiative_id);
CREATE INDEX IF NOT EXISTS idx_initiative_program_info_terms_iid ON
initiative_program_info_terms(initiative_id);
CREATE INDEX IF NOT EXISTS idx_initiative_program_info_skills_iid ON
initiative_program_info_skills(initiative_id);
CREATE INDEX IF NOT EXISTS idx_initiative_sponsorship_tiers_iid ON
initiative_sponsorship_tiers(initiative_id);
CREATE INDEX IF NOT EXISTS idx_initiative_contacts_iid ON
initiative_contacts(initiative_id);

```

```

-- donations
CREATE INDEX IF NOT EXISTS idx_donations_user_id ON
donations(user_id);
CREATE INDEX IF NOT EXISTS idx_donations_initiative_id ON
donations(initiative_id);
CREATE INDEX IF NOT EXISTS idx_donations_status ON
donations(status);
CREATE INDEX IF NOT EXISTS idx_donations_org_id ON
donations(organization_id);

-- subscriptions
CREATE INDEX IF NOT EXISTS idx_subscriptions_user_id ON
subscriptions(user_id);
CREATE INDEX IF NOT EXISTS idx_subscriptions_initiative_id ON
subscriptions(initiative_id);
CREATE INDEX IF NOT EXISTS idx_subscriptions_org_id ON
subscriptions(organization_id);
CREATE INDEX IF NOT EXISTS idx_subscriptions_status ON
subscriptions(status);

COMMIT;

```

Key notes:

- Wrapped in a **BEGIN/COMMIT** transaction — if anything fails, the entire migration rolls back cleanly
- **CREATE TABLE IF NOT EXISTS** makes it safe to run without crashing if tables already exist
- **DEFAULT NOW()** added to **created_on** and **updated_on** for automatic timestamping

Normalization summary

None

Normalised Initiatives Schema – Reference Document

****Last updated:**** 2026-05-11

****Source:**** Live scan of `lff-prod-entities` (182 records) and `lff-prod-projects` (1,841 records)

****Schema version:**** 2.0.0

****Migration status:**** :white_check_mark: Complete – 2,021 initiatives, 16,840 users, 1,598 donations, 277 subscriptions migrated

Overview

Schema v2.0.0 fully normalises all DynamoDB `lff-prod-\*` tables into 20 PostgreSQL tables.

The two source initiative tables (`lff-prod-projects` and `lff-prod-entities`) are merged into

a single `initiatives` table with 15 child tables handling all repeating groups and type-specific

detail. No JSONB columns survive on core entity tables (except `cached\_details` on donations /

subscriptions, and `entity\_details` as a catch-all for free-form application metadata).

Table inventory

...

users	- lff-prod-users
organizations	- lff-prod-organizations
initiatives	- lff-prod-projects + lff-prod-entities
(merged)	
├─ initiative_goals	- budget categories (projects) + goals[]
(entities)	
├─ initiative_beneficiaries	- beneficiaries[] (both)
├─ initiative_custom_websites	- customWebsites[] (both)
├─ initiative_contributors	- contributors[] (projects only)
├─ initiative_mentors	- programInfo.mentor[] (projects only)
├─ initiative_program_info_terms	- programInfo.terms[] (projects only)

```

    |— initiative_program_info_skills - programInfo.skills[] (projects only)
    |— initiative_program_info_config - programInfo.termsConditions bool (projects
only)
    |— initiative_program_info_custom_term - programInfo.customTerm (projects only)
    |— initiative_sponsorship_tiers - sponsorshipTiers[] (entities only)
    |— initiative_ostif_detail - detail struct (ostif entities only)
    |— initiative_contacts - detail.{primary,secondary,technicalLead}
(ostif only)
    |— initiative_github_stats - cachedDetails.githubStats (projects only)
    |— initiative_stats - cachedDetails.projectStats.backers
(projects only)
    |— initiative_entity_details - entityDetails map[string]string (entities
only)
donations - lff-prod-donations +
lff-prod-entity-donations (merged)
subscriptions - lff-prod-subscriptions +
lff-prod-entity-subscriptions (merged)
...

```

Source → target field mapping

`initiatives` (core columns)

```

| PostgreSQL column | Project source (`lff-prod-projects`) | Entity source
(`lff-prod-entities`) | Go type |
|---|---|---|---|
| `initiative_id` | `projectId` | `entityId` | `string` |
| `initiative_type` | hardcoded `project` | `entityType` (see quirk below) |
`string` |
| `source_dynamo_table` | `projects` | `entities` | migration-only |
| `owner_id` | `ownerId` | `ownerId` | `string` |
| `name` | `name` | `name` | `string` |
| `slug` | `slug` | `slug` | `string` |
| `status` | `status` | `status` | `string` - entity: `pending\|published` |
| `industry` | `projectDetails.industry` | `industry` | `string` |
| `description` | `projectDetails.description` | `description` | `string` |
| `color` | `projectDetails.color` | `color` | `string` - hex, max 7 chars |
| `logo_url` | `logoUrl` | `logoUrl` | `string` |
| `website_url` | `projectDetails.website` | `websiteURL` | `string` |
| `coc_url` | `projectDetails.codeOfConduct.link` | `cocURL` | `string` |
| `cii_project_id` | `projectDetails.ciiProjectID` | - | `string` (full badge URLs
observed) |
| `stripe_plan_id` | `planId` | `stripePlanId` | `string` |

```

```

| `stripe_product_id` | `productId` | `stripeProductId` | `string` |
| `jobspring_project_id` | `jobspringProjectId` | - | `string` |
| `stacks_identifider` | `projectDetails.stacksIdentifider` | - | `string` |
| `eventbrite_id` | - | `eventbriteId` | `string` |
| `application_url` | - | `applicationURL` | `string` |
| `amount_raised` | `amountRaised` | `amountRaised` | `int` |
| `accept_funding` | `false` (hardcoded) | `acceptFunding` | `bool` |
| `event_start_date` | - | `eventStartDate` | `string` → `TIMESTAMP TZ` |
| `event_end_date` | - | `eventEndDate` | `string` → `TIMESTAMP TZ` |
| `country` | - | `EntityLocation.country` | `string` |
| `city` | - | `EntityLocation.city` | `string` |
| `is_online` | `false` (hardcoded) | `isOnline` | `bool` |
| `created_on` | `createdOn` | `createdOn` | `string` → `TIMESTAMP TZ` |
| `updated_on` | `updatedOn` | `updatedOn` | `string` → `TIMESTAMP TZ` |

**entityType quirk:** `SaveEntity` rewrites `general fund` → `initiative`
before every
`PutItem`. Migration reverses this: `initiative` → `general fund`.

### `initiative_goals`

| Column | Projects source | Entities source |
|---|---|---|
| `name` | category key: `development`, `marketing`, `meetups`, `travel`,
`bugBounty`, `documentation`, `other`, `mentee` | `goals[].name` |
| `amount_in_cents` | `projectDetails.<cat>.budget.amount` (JSON tag `"amount"`,
not `"amountInCents"`) | `goals[].budget.amount` |
| `allocation` | `projectDetails.<cat>.budget.allocation` |
`goals[].budget.allocation` |
| `repo_link` | `projectDetails.development.repoLink` | - |
| `description` | - | `goals[].description` |
| `color` | - | `goals[].goalColor` |
| `icon` | - | `goals[].goalIcon` |
| `sort_order` | fixed: development=0 ... mentee=7 | array index |

### `initiative_mentors`
Source: `projectDetails.programInfo.mentor[]` - `Mentor{name, email, avatarURL,
introduction}`.

### `initiative_program_info_terms` / `_skills`
Source: `projectDetails.programInfo.terms[]` and `_skills[]` - both `[]string`.

### `initiative_program_info_config`
Source: `projectDetails.programInfo.termsConditions` - `bool`.

```

```

### `initiative_program_info_custom_term`
Source: `projectDetails.programInfo.customTerm` - `CustomTerm{termName,
startMonth, endMonth, year}`.
Row only inserted when `termName` is non-empty.

### `initiative_sponsorship_tiers`
Source: `entity.sponsorshipTiers[]` - `SponsorshipTier{name, description, color,
icon, minimum}`.

### `initiative_ostif_detail`
Source: `entity.detail` (cast to `domain.Detail`) when `entityType == 'ostif'`.
Fields: `monetizationStrategy`, `currentSecurityStrategy`, `licenseType`,
`totalBudget` (`int`), `termsConditions` (`bool`).

### `initiative_contacts`
Source: `entity.detail.{primaryContact, secondaryContact, technicalLead}` -
`Contact{firstName, lastName, email, phoneNumber, otherContactOption,
preferredContactMethod}`.
`contact_type` values: `'primary'`, `'secondary'`, `'technical_lead'`.

### `initiative_github_stats`
Source: `project.cachedDetails.githubStats` - `GithubStats{forks, stars,
openIssues}` (all `int`).

### `initiative_stats`
Source: `project.cachedDetails.projectStats.backers` (`int`).
`totalRaised` excluded - no active write path (always 0 in production).

### `initiative_entity_details`
Source: `entity.entityDetails` - `map[string]string`, serialised as JSONB.
All production records have empty maps; table exists for completeness.

### `donations`

| Column | `lff-prod-donations` | `lff-prod-entity-donations` |
|---|---|---|
| `user_id` | `userId` | `userId` |
| `initiative_id` (UUID FK → `initiatives.id`) | `projectId` → `_as_uuid()` |
| `entityId` → `_as_uuid()` |
| `organization_id` | `orgId` → `_as_uuid()` or NULL | same |
| `category` | `category` | `category` |
| `created_on` | `createdOn` | `createdOn` |
| `current_amount_in_cents` | `currentAmountInCents` | `currentAmountInCents` |
| `payment_method` | `paymentmethod` | `paymentmethod` |
| `status` | `status` | `status` |

```

```
| `stripe_charge_id` | `stripeChargeId` | `stripeChargeId` |  
| `cached_details` | `cachedDetails` JSONB | same |
```

```
### `subscriptions`
```

```
Mirrors `donations` structure. Additional columns: `frequency`,  
`stripe_subscription_id`, `stripe_subscription_item_id`.
```

```
---
```

```
## Column type rationale
```

```
| Type | Used for | Reason |  
|---|---|---|  
| `TEXT` | All free-form string fields | No length enforced in Go; real data  
exceeds 255 chars (CII URLs, descriptions, skill names) |  
| `VARCHAR(255)` | Natural keys (`user_id`, `initiative_id`, `owner_id`) and  
Stripe/external IDs that are structurally bounded | Preserves index efficiency for  
FK join columns |  
| `VARCHAR(50)` | Enum-like fields: `status`, `initiative_type`, `payment_method`,  
`frequency`, `contact_type` | Go validation constraints (`valid:"in(...)"`)  
enforce short values |  
| `VARCHAR(10)` | `color` | Hex color max 7 chars (`#RRGGBB`); `valid:"hexcolor"`  
enforced in Go |  
| `BIGINT` | `current_amount_in_cents` (donations, subscriptions),  
`amount_in_cents` (goals), `total_budget` (ostif) | Go uses `int64` for monetary  
amounts |  
| `INTEGER` | `amount_raised`, stats counters | Go uses `int`; max observed value  
~9.8M (fits 32-bit) |  
| `BOOLEAN` | `accept_funding`, `is_online`, `terms_conditions` | Go `bool` |  
| `TIMESTAMPTZ` | All timestamps | DynamoDB stores as strings in multiple formats;  
normalised to UTC on import |  
| `UUID` | Surrogate PKs, `initiatives.id`, `organizations.id`, FK refs in  
donations/subscriptions | Stable across re-runs via deterministic `uuid5` |  
| `JSONB` | `donations.cached_details`, `subscriptions.cached_details`,  
`initiative_entity_details.details` | Variable structure; not queried by key in  
application hot path |
```

```
---
```

```
## Fields excluded from schema
```

Field	Reason
`balance`	Computed from ledger API at read time; never written to DynamoDB directly
`funding_status`	Computed from ledger API + subscription summaries at read time
`entity_stats.sponsors`	`GetEntitySponsors` pushes to Elasticsearch only; `SaveEntity` never called with sponsors populated
`diversity`	`GetDiversity` fetches external API at read time
`vulnerability_summary`	`GetVulnerabilitySummary` fetches external service at read time
`badges`	`convertProjectToDynamoRepresentation` never assigns `Badges`
`projectStats.totalRaised`	No active write path – always 0 in production
`sponsors[]`	Read-time only; populated by `GetEntitySponsors` via ES

Financial data: balance and funding_status

Why they are excluded

`balance` and `funding\_status` are not stored in the PostgreSQL schema because they are

****never persisted**** – neither in DynamoDB nor anywhere else. They are computed fresh on every

API call from two live sources:

1. ****Ledger API**** (`balanceData` / `ledgerService`) – an external service that holds the canonical transaction ledger for each initiative. All credits (donations, subscriptions) and debits (payouts to beneficiaries) live there.
2. ****Subscription summaries**** (`subscriptionRepository.GetSubscriptionSummary`) – aggregated directly from the `lff-prod-subscriptions` / `lff-prod-entity-subscriptions` tables (now the `subscriptions` table in PostgreSQL).

The DynamoDB records for `balance` and `funding\_status` were snapshots written by background

jobs (`updateEntityAmountRaised`, `seed\_amountraised\_\*`) and are ****stale by design**** – they

existed solely as a read-cache for list views and are now superseded by the PostgreSQL

`amount\_raised` column and real-time derivation described below.

Go data model

```
```go
// Balance - sourced entirely from ledger API
type Balance struct {
 BalanceInCents int // credit - debit
 AvailableInCents int // balance after
 reserved amounts
 CreditInCents int // total donations
 received
 DebitInCents int // total payouts made
 AmountRaisedLastYear int
 AmountRaisedCurrentYear int
 Expenditures map[ProjectCategory]Expenditure // per-category
 breakdown
}
```

```
type Expenditure struct {
 BalanceInCents int // CreditInCents - DebitInCents for this category
 CreditInCents int // donations allocated to this category
 DebitInCents int // payouts from this category
 TransactionsCount int
}
```

```
// FundingStatus - ledger API + subscription summaries
type FundingStatus struct {
 TotalAnnualGoalInCents int64 // sum of
 initiative_goals.amount_in_cents
 TotalDonationsInCents int64 // balance.CreditInCents
 TotalDebitsInCents int64 // balance.DebitInCents
 TotalSubscriptionCount int // subscriptions aggregate
 AnnualSubscriptionAmountInCents int64 // subscriptions aggregate
 AnnualSubscriptionRemainingAmountInCents int64 // goal - subscribed amount
}
```
```

How each value is derived per initiative

`balance.credit\_in\_cents` - total donations received

```
```sql
-- All-time credited amount for an initiative from the PostgreSQL data
SELECT
```

```

 i.initiative_id,
 i.name,
 COALESCE(SUM(d.current_amount_in_cents), 0) AS credit_in_cents
FROM initiatives i
LEFT JOIN donations d ON d.initiative_id = i.id
 AND d.status = 'completed'
GROUP BY i.initiative_id, i.name;
```

```

> The ledger API credits both one-off donations and settled subscription payments. The

> PostgreSQL `donations` table holds one-off charges; `subscriptions` holds recurring ones.

> For a full credit total: `SUM(donations) + SUM(settled subscription payments)`.

`balance.debit\_in\_cents` – payouts to beneficiaries

Debits originate from the `lff-prod-transactions` table (payout records) which is **not yet**

migrated to PostgreSQL. Once `transactions` is migrated, debits can be derived as:

```

```sql
SELECT
 initiative_id,
 SUM(amount_in_cents) AS debit_in_cents
FROM transactions
WHERE transaction_type = 'debit'
GROUP BY initiative_id;
```

```

`balance.balance\_in\_cents` – net available

```

...
balance_in_cents = credit_in_cents - debit_in_cents
...

```

`funding\_status.total\_annual\_goal\_in\_cents` – budget target

Directly derivable from the normalised schema:

```

```sql
-- Projects: sum of all budget category goals
SELECT
 initiative_id,

```

```

 SUM(amount_in_cents) AS total_annual_goal_in_cents
FROM initiative_goals
GROUP BY initiative_id;
```

```

For entities the total goal is stored explicitly in `entity.FundingStatus.TotalAnnualGoalInCents` (set by `EditEntityRequest.Detail.TotalBudget` for ostif entities, or summed from goals for others). This value is already captured in `initiative\_ostif\_detail.total\_budget` for ostif entities; for non-ostif entities it can be recomputed from `initiative\_goals`.

`funding\_status.total\_subscription\_count` and `annual\_subscription\_amount\_in\_cents`

Directly derivable from the `subscriptions` table:

```

```sql
SELECT
 i.id AS initiative_pg_id,
 i.initiative_id,
 COUNT(s.id) AS total_subscription_count,
 COALESCE(SUM(s.current_amount_in_cents), 0) AS
annual_subscription_amount_in_cents
FROM initiatives i
LEFT JOIN subscriptions s ON s.initiative_id = i.id
 AND s.status = 'active'
GROUP BY i.id, i.initiative_id;
```

```

Per-category funding status (projects only)

For the `ProjectFundingStatus.{Development, Marketing, ...}` breakdown, join goals to subscription `category`:

```

```sql
SELECT
 g.initiative_id,
 g.name AS category,
 g.amount_in_cents AS goal_in_cents,
 COALESCE(SUM(d.current_amount_in_cents), 0) AS donations_in_cents,
 COUNT(s.id) AS subscription_count,

```

```

 COALESCE(SUM(s.current_amount_in_cents), 0) AS
subscription_amount_in_cents
FROM initiative_goals g
JOIN initiatives i ON i.initiative_id = g.initiative_id
LEFT JOIN donations d ON d.initiative_id = i.id AND d.category = g.name AND
d.status = 'completed'
LEFT JOIN subscriptions s ON s.initiative_id = i.id AND s.category = g.name AND
s.status = 'active'
GROUP BY g.initiative_id, g.name, g.amount_in_cents;
```

```

Recommended PostgreSQL view

A materialised view over the above queries avoids recomputing on every API call and removes the dependency on the external ledger API for read-path queries (once transactions are migrated):

```

```sql
CREATE MATERIALIZED VIEW initiative_funding_summary AS
SELECT
 i.id AS
initiative_pg_id,
 i.initiative_id,
 i.name,
 COALESCE(SUM(DISTINCT g.amount_in_cents), 0) AS
total_annual_goal_in_cents,
 COALESCE(SUM(d.current_amount_in_cents)
 FILTER (WHERE d.status = 'completed'), 0) AS
total_donations_in_cents,
 COUNT(s.id) FILTER (WHERE s.status = 'active') AS
total_subscription_count,
 COALESCE(SUM(s.current_amount_in_cents)
 FILTER (WHERE s.status = 'active'), 0) AS
annual_subscription_amount_in_cents
FROM initiatives i
LEFT JOIN initiative_goals g ON g.initiative_id = i.initiative_id
LEFT JOIN donations d ON d.initiative_id = i.id
LEFT JOIN subscriptions s ON s.initiative_id = i.id
GROUP BY i.id, i.initiative_id, i.name;

-- Refresh after each donation/subscription write:
-- REFRESH MATERIALIZED VIEW CONCURRENTLY initiative_funding_summary;
```

```

```
> **Note:** `balance.debit_in_cents` cannot be populated until  
`lff-prod-transactions` is  
> migrated to a `transactions` table. Until then, the ledger API remains the  
authoritative  
> source for payout data and `balance_in_cents`.
```

Migration statistics (production data - 2026-05-11)

| Table | Rows |
|-------------------------------------|--------|
| users | 16,840 |
| organizations | 606 |
| initiatives | 2,021 |
| initiative_goals | 4,436 |
| initiative_beneficiaries | 2,004 |
| initiative_contributors | 37 |
| initiative_custom_websites | 1 |
| initiative_mentors | 2,911 |
| initiative_program_info_skills | 5,462 |
| initiative_program_info_terms | 92 |
| initiative_program_info_config | 1,840 |
| initiative_program_info_custom_term | 11 |
| initiative_sponsorship_tiers | 0 |
| initiative_ostif_detail | 11 |
| initiative_contacts | 26 |
| initiative_github_stats | 1,551 |
| initiative_stats | 1,840 |
| initiative_entity_details | 0 |
| donations | 1,598 |
| subscriptions | 277 |

****FK integrity:**** 0 orphaned goals, 0 orphaned donations (initiative or user), 0 NULL owner IDs.

****Initiative type breakdown:****

| initiative_type | Source | Count |
|-----------------|----------|-------|
| project | projects | 1,839 |
| general fund | entities | 122 |
| other | entities | 26 |
| event | entities | 20 |
| ostif | entities | 11 |

```
| `community` | entities | 3 |
```

```
---
```

```
## Post-cutover cleanup (Phase 5)
```

Once the application no longer writes to DynamoDB, drop the migration scaffolding:

```
```sql
```

```
ALTER TABLE initiatives DROP COLUMN source_dynamo_table;
```

```
```
```

Data Migration Script

Python

```
#!/usr/bin/env python3
# Copyright The Linux Foundation and each contributor to CommunityBridge.
# SPDX-License-Identifier: MIT
"""
DynamoDB → PostgreSQL Migration Script
=====
Migrates all lff-prod-* DynamoDB tables into the PostgreSQL schema defined
in db/schema.sql (v2 – fully normalised initiatives schema).
Source → Target mapping
-----
lff-prod-users                → users
lff-prod-organizations        → organizations
lff-prod-projects + lff-prod-entities → initiatives (merged)
                                → initiative_goals
                                → initiative_beneficiaries
                                → initiative_custom_websites
                                → initiative_contributors
                                → initiative_mentors
                                → initiative_program_info_terms
                                → initiative_program_info_skills
                                → initiative_program_info_config
                                →
initiative_program_info_custom_term
                                → initiative_sponsorship_tiers
                                → initiative_ostif_detail
                                → initiative_contacts
                                → initiative_github_stats
                                → initiative_stats
                                → initiative_entity_details
lff-prod-donations + lff-prod-entity-donations → donations      (merged)
lff-prod-subscriptions
+ lff-prod-entity-subscriptions                → subscriptions (merged)
Key notes
-----
- initiatives.initiative_id = projectId (projects) or entityId (entities)
  stored as VARCHAR(255) – not cast to UUID; the column is a natural key.
- initiatives.id (UUID surrogate PK) is generated deterministically via
  _as_uuid(projectId / entityId) so FK lookups from donations/subscriptions
  remain stable across re-runs.
```

- DynamoDB entity type quirk: SaveEntity rewrites 'general fund' → 'initiative' before every PutItem. Migration reverses this:
 - if entityType == 'initiative' → restore to 'general fund'
- Budget.AmountInCents is serialised with json tag "amount" – access as budget.get("amount"), NOT budget.get("amountInCents").
- Excluded from schema (no DynamoDB write path / computed at read-time):
 - balance, funding_status, entity_stats, sponsors, diversity, vulnerability_summary, badges, projectStats.totalRaised
- All INSERTs use ON CONFLICT ... DO UPDATE (idempotent; safe to re-run).

Usage

```
-----
export AWS_ACCESS_KEY_ID=...
export AWS_SECRET_ACCESS_KEY=...
export AWS_SESSION_TOKEN=...          # for temporary/STS credentials
export AWS_REGION=us-east-1
export PG_DSN="host=localhost port=5432 dbname=lf user=postgres password=..."
python3 migrate_dynamo_to_postgres.py
```

Dependencies

```
-----
pip install boto3 psycopg2-binary
"""

import json
import logging
import os
import sys
import uuid
from decimal import Decimal
import boto3
import psycopg2
import psycopg2.extras
from boto3.dynamodb.types import TypeDeserializer as _TypeDeserializer

# -----
# Logging
# -----
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
log = logging.getLogger(__name__)

# -----
# Configuration
# -----
REGION = os.environ.get("AWS_REGION", "us-east-1")
PG_DSN = os.environ.get(
```

```

    "PG_DSN",
    "host=localhost port=5432 dbname=lfv user=postgres password=postgres",
)
# Stable UUID namespace - must not change between runs to keep IDs deterministic
_UUID_NS = uuid.UUID("6ba7b810-9dad-11d1-80b4-00c04fd430c8")
# -----
# DynamoDB helpers
# -----
_deser = _TypeDeserializer()
def _deserialize(item: dict) -> dict:
    """Convert DynamoDB typed attribute map → plain Python dict."""
    return {k: _deser.deserialize(v) for k, v in item.items()}
def scan_table(client, table_name: str) -> list:
    """Full table scan with automatic pagination."""
    log.info("Scanning %-48s", table_name + " ...")
    items: list = []
    kwargs: dict = {"TableName": table_name}
    while True:
        resp = client.scan(**kwargs)
        items.extend(_deserialize(raw) for raw in resp.get("Items", []))
        lek = resp.get("LastEvaluatedKey")
        if not lek:
            break
        kwargs["ExclusiveStartKey"] = lek
    log.info(" → %d items", len(items))
    return items
# -----
# General helpers
# -----
def _uuid5(scope: str, *parts) -> str:
    key = "|".join(str(p) for p in parts)
    return str(uuid.uuid5(_UUID_NS, f"{scope}:{key}"))
def _as_uuid(value) -> str | None:
    """Return a valid UUID string or None; coerce non-UUID strings via uuid5."""
    if value is None:
        return None
    s = str(value).strip()
    if not s:
        return None
    try:
        return str(uuid.UUID(s))
    except ValueError:
        return _uuid5("coerce", s)
def _as_int(value, default: int = 0) -> int:
    if value is None:

```

```

        return default
    if isinstance(value, Decimal):
        return int(value)
    try:
        return int(value)
    except (TypeError, ValueError):
        return default
def _to_jsonb(value) -> str | None:
    if value is None:
        return None
    def _default(obj):
        if isinstance(obj, Decimal):
            return float(obj)
        raise TypeError(f"Cannot serialize {type(obj)}")
    return json.dumps(value, default=_default)
def _parse_ts(s: str | None):
    """Parse DynamoDB timestamp string → tz-aware datetime (or None).
    Handles:
        - Standard formats: '2024-05-07 09:53:16 +0000', '2023-12-01T20:31:33Z'
        - Go time.String() format with nanoseconds + monotonic suffix:
          '2019-04-02 15:42:26.518360269 +0000 UTC m=+2580.502337766'
    """
    if not s:
        return None
    import re
    from datetime import datetime, timezone
    # Strip Go monotonic clock suffix: ' UTC m=+...' or ' UTC m=-...'
    cleaned = re.sub(r'\s+UTC\s+m=[+-][\d.]+\$', '', s)
    # Truncate sub-second precision to at most 6 digits (microseconds)
    cleaned = re.sub(r'(\.\d{6})\d+', r'\1', cleaned)
    for fmt in (
        "%Y-%m-%d %H:%M:%S.%f %Z",
        "%Y-%m-%d %H:%M:%S %Z",
        "%Y-%m-%dT%H:%M:%S.%f%Z",
        "%Y-%m-%dT%H:%M:%S%Z",
        "%Y-%m-%d %H:%M:%S.%f",
        "%Y-%m-%d %H:%M:%S",
        "%Y-%m-%dT%H:%M:%S",
        "%Y-%m-%d",
    ):
        try:
            dt = datetime.strptime(cleaned, fmt)
            if dt.tzinfo is None:
                dt = dt.replace(tzinfo=timezone.utc)
            return dt

```

```

        except ValueError:
            continue
        log.warning("Could not parse timestamp: %r", s)
        return None
def _trunc(value: str | None, max_len: int, label: str = "") -> str | None:
    """Truncate a string to max_len characters, warning when it happens."""
    if value is None:
        return None
    if len(value) > max_len:
        log.warning("Truncating %s value (%d chars → %d): %r...", label, len(value),
max_len, value[:60])
        return value[:max_len]
    return value
# -----
# Migration: user
# -----
def migrate_users(cur, users: list, placeholder_ids: set) -> set:
    """
    Upsert users from lff-prod-users.
    Insert placeholder rows for any user_id referenced elsewhere but absent here.
    Returns the complete set of known user_ids after migration.
    """
    log.info("Migrating users (%d records + %d placeholders) ...", len(users),
len(placeholder_ids))
    sql = """
        INSERT INTO users (user_id, email, given_name, family_name, name,
avatar_url)
        VALUES (%s, %s, %s, %s, %s, %s)
        ON CONFLICT (user_id) DO UPDATE SET
            email      = EXCLUDED.email,
            given_name = EXCLUDED.given_name,
            family_name = EXCLUDED.family_name,
            name       = EXCLUDED.name,
            avatar_url = EXCLUDED.avatar_url
    """
    rows: list = []
    known: set = set()
    for u in users:
        uid = u.get("id")
        if not uid:
            continue
        known.add(uid)
        rows.append((uid, u.get("email"), u.get("givenName"), u.get("familyName"),
u.get("name"), u.get("avatarUrl")))
    for uid in placeholder_ids:

```

```

        if uid and uid not in known:
            known.add(uid)
            rows.append((uid, None, None, None, None, None))
        psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
        log.info(" → %d user rows upserted", len(rows))
        return known

# -----
# Migration: organization
# -----

def migrate_organizations(cur, orgs: list, known_users: set) -> set:
    """
    Upsert organizations from lff-prod-organizations.
    Returns the set of postgres organization UUIDs inserted (for FK resolution).
    """
    log.info("Migrating organizations (%d records) ...", len(orgs))
    sql = """
        INSERT INTO organizations (id, organization_id, owner_id, name,
avatar_url, status)
        VALUES (%s, %s, %s, %s, %s, %s)
        ON CONFLICT (id) DO UPDATE SET
            organization_id = EXCLUDED.organization_id,
            owner_id       = EXCLUDED.owner_id,
            name           = EXCLUDED.name,
            avatar_url     = EXCLUDED.avatar_url,
            status         = EXCLUDED.status
    """
    rows: list = []
    known_org_ids: set = set()
    skipped = 0
    for o in orgs:
        owner_id = o.get("ownerId")
        if owner_id not in known_users:
            skipped += 1
            log.debug("Skip org %s - owner %s not found", o.get("organizationId"),
owner_id)
            continue
        pg_id = _as_uuid(o.get("organizationId")) or str(uuid.uuid4())
        known_org_ids.add(pg_id)
        rows.append((pg_id, pg_id, owner_id, o.get("name"), o.get("avatarUrl"),
o.get("status")))
    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    if skipped:
        log.warning(" → %d organization(s) skipped (owner not found)", skipped)
    log.info(" → %d organization rows upserted", len(rows))
    return known_org_ids

```

```

# -----
# Project goal category ordering (matches convertProjectToDynamoRepresentation)
# -----
_PROJECT_GOAL_CATEGORIES = [
    ("development", "development", 0),
    ("marketing", "marketing", 1),
    ("meetups", "meetups", 2),
    ("travel", "travel", 3),
    ("bugBounty", "bugBounty", 4),
    ("documentation", "documentation", 5),
    ("other", "other", 6),
    # 'mentee' is handled separately (sort_order 7)
]

# -----
# Migration: initiatives (merged from lff-prod-entities + lff-prod-projects)
# -----
def migrate_initiatives(cur, entities: list, projects: list, known_users: set) ->
set:
    """
    Merge lff-prod-entities and lff-prod-projects into the initiatives table
    and all normalised child tables.
    Projects → initiative_type = 'project'; initiative_id = projectId
    Entities → initiative_type from entityType; initiative_id = entityId
    QUIRK: DynamoDB stores 'general fund' as 'initiative'.
            Migration restores it: 'initiative' → 'general fund'.
    Returns the set of PostgreSQL initiative UUIDs (initiatives.id) that were
    inserted – used as FK targets for the donations/subscriptions migration.
    """
    log.info(
        "Migrating initiatives (%d entities + %d projects = %d total) ...",
        len(entities), len(projects), len(entities) + len(projects),
    )
    # — Row buffers (one list per target table) —————
    initiative_rows: list = []
    goals_rows: list = []
    beneficiaries_rows: list = []
    custom_websites_rows: list = []
    contributors_rows: list = []
    mentors_rows: list = []
    program_info_terms_rows: list = []
    program_info_skills_rows: list = []
    program_info_config_rows: list = []
    program_info_custom_term_rows: list = []
    sponsorship_tiers_rows: list = []
    ostif_detail_rows: list = []

```

```

contacts_rows:          list = []
github_stats_rows:      list = []
stats_rows:             list = []
entity_details_rows:    list = []
known_initiative_ids:   set = set()
skipped = 0
# — Projects —————
for p in projects:
    owner_id = p.get("ownerId")
    if owner_id not in known_users:
        skipped += 1
        log.debug("Skip project %s – owner %s not found", p.get("projectId"),
owner_id)
        continue
    raw_initiative_id = p.get("projectId") or ""
    if not raw_initiative_id:
        skipped += 1
        log.warning("Skip project with no projectId (name=%s)", p.get("name"))
        continue
    pg_id = _as_uuid(raw_initiative_id) or _uuid5("project", owner_id,
p.get("name", ""))
    known_initiative_ids.add(pg_id)
    # projectDetails nested map (ProjectDetails struct serialised via JSON
tags)
    pd: dict = p.get("projectDetails") or {}
    # cachedDetails nested map (CachedDetails struct serialised via JSON tags)
    cd: dict = p.get("cachedDetails") or {}
    coc = pd.get("codeOfConduct")
    coc_url = coc.get("link") if isinstance(coc, dict) else None
    # — Core initiative row —————
    initiative_rows.append((
        pg_id,                                # id
        raw_initiative_id,                    # initiative_id
        "project",                            # initiative_type
        "projects",                          # source_dynamo_table
        owner_id,                             # owner_id
        p.get("name"),                        # name
        p.get("slug"),                       # slug
        p.get("status"),                     # status
        pd.get("industry"),                  # industry
        pd.get("description"),               # description
        _trunc(pd.get("color"), 10, "color"), # color
        p.get("logoUrl"),                   # logo_url
        pd.get("website"),                   # website_url

```

```

        coc_url,                                # coc_url
        pd.get("ciiProjectID"),                 # cii_project_id
        p.get("planId"),                        # stripe_plan_id
        p.get("productId"),                    # stripe_product_id
        p.get("jobspringProjectId") or None,    # jobspring_project_id
        pd.get("stacksIdentifier") or None,     # stacks_identifier
        None,                                  # eventbrite_id (entity
only)
        None,                                  # application_url (entity
only)
        _as_int(p.get("amountRaised")),         # amount_raised
        False,                                 # accept_funding (entity
only)
        None,                                  # event_start_date
(entity only)
        None,                                  # event_end_date
(entity only)
        None,                                  # country (entity only)
        None,                                  # city (entity only)
        False,                                 # is_online (entity only)
        _parse_ts(p.get("createdOn")),         # created_on
        _parse_ts(p.get("updatedOn")),         # updated_on
    ))
# — Budget category goals —————
for cat_key, goal_name, sort_order in _PROJECT_GOAL_CATEGORIES:
    cat: dict = pd.get(cat_key) or {}
    budget: dict = cat.get("budget") or {}
    # Budget.AmountInCents is serialised with json tag "amount"
    amt = _as_int(budget.get("amount"))
    alloc = budget.get("allocation") or None
    repo = cat.get("repoLink") or None if cat_key == "development" else
None

    if amt > 0 or alloc or repo:
        goals_rows.append((
            _uuid5("goal", raw_initiative_id, goal_name),
            raw_initiative_id, goal_name, amt, alloc, repo,
            None, None, None, # description, color, icon (entity only)
            sort_order,
        ))
# — Mentee as a special goal (sort_order 7) —————
mentee: dict = pd.get("mentee") or {}
if mentee:
    m_budget: dict = mentee.get("budget") or {}
    m_amt = _as_int(m_budget.get("amount"))
    m_alloc = m_budget.get("allocation") or None

```

```

        if m_amt > 0 or m_alloc or mentee.get("mentor") or mentee.get("terms")
or mentee.get("skills"):
            goals_rows.append((
                _uuid5("goal", raw_initiative_id, "mentee"),
                raw_initiative_id, "mentee", m_amt, m_alloc, None,
                None, None, None,
                7,
            ))
    # Mentors
    for idx, mentor in enumerate(mentee.get("mentor") or []):
        email = mentor.get("email") or None
        mentors_rows.append((
            _uuid5("mentor", raw_initiative_id, email or
mentor.get("name", "") or str(idx)),
            raw_initiative_id,
            mentor.get("name"),
            email,
            mentor.get("avatarURL") or None,
            mentor.get("introduction") or None,
        ))
    # Mentee config (termsConditions bool – always present once mentee is
set)
    program_info_config_rows.append((
        raw_initiative_id,
        bool(mentee.get("termsConditions", False)),
    ))
    # Mentee terms ([]string)
    for idx, term in enumerate(mentee.get("terms") or []):
        if term:
            program_info_terms_rows.append((
                _uuid5("mentee_term", raw_initiative_id, str(idx)),
                raw_initiative_id, str(term), idx,
            ))
    # Mentee skills ([]string)
    for skill in (mentee.get("skills") or []):
        if skill:
            program_info_skills_rows.append((
                _uuid5("mentee_skill", raw_initiative_id, str(skill)),
                raw_initiative_id, str(skill),
            ))
    # Mentee custom term – only when termName is non-empty
    ct: dict = mentee.get("customTerm") or {}
    if isinstance(ct, dict) and ct.get("termName"):
        program_info_custom_term_rows.append((
            raw_initiative_id,

```

```

        ct.get("termName"),
        ct.get("startMonth") or None,
        ct.get("endMonth") or None,
        _as_int(ct.get("year")) or None,
    ))

# — Contributors —————
for idx, c in enumerate(pd.get("contributors") or []):
    email = c.get("email") or None
    contributors_rows.append((
        _uuid5("contributor", raw_initiative_id, email or c.get("name",
"")) or str(idx)),
        raw_initiative_id, c.get("name"), email,
    ))

# — Beneficiaries —————
for idx, b in enumerate(pd.get("beneficiaries") or []):
    email = b.get("email") or None
    beneficiaries_rows.append((
        _uuid5("beneficiary", raw_initiative_id, email or b.get("name",
"")) or str(idx)),
        raw_initiative_id, b.get("name"), email,
    ))

# — Custom websites —————
for cw in (pd.get("customWebsites") or []):
    url = cw.get("url") or ""
    if url:
        custom_websites_rows.append((
            _uuid5("custom_website", raw_initiative_id, url),
            raw_initiative_id, cw.get("name") or None, url,
        ))

# — GitHub stats —————
gh: dict = cd.get("githubStats") or {}
if gh.get("forks") or gh.get("stars") or gh.get("openIssues"):
    github_stats_rows.append((
        raw_initiative_id,
        _as_int(gh.get("forks")),
        _as_int(gh.get("stars")),
        _as_int(gh.get("openIssues")),
    ))

# — Project stats (backers only - totalRaised excluded) —————
ps: dict = cd.get("projectStats") or {}
stats_rows.append((
    raw_initiative_id,
    _as_int(ps.get("backers")),
))

# — Entities —————

```

```

for e in entities:
    owner_id = e.get("ownerId")
    if owner_id not in known_users:
        skipped += 1
        log.debug("Skip entity %s - owner %s not found", e.get("entityId"),
owner_id)
        continue
    raw_initiative_id = e.get("entityId") or ""
    if not raw_initiative_id:
        skipped += 1
        log.warning("Skip entity with no entityId (name=%s)", e.get("name"))
        continue
    pg_id = _as_uuid(raw_initiative_id) or _uuid5("entity", owner_id,
e.get("name", ""))
    known_initiative_ids.add(pg_id)
    # entityType quirk: SaveEntity rewrites 'general fund' → 'initiative'
    # before every PutItem. Reverse it here.
    entity_type: str = e.get("entityType") or ""
    if entity_type == "initiative":
        entity_type = "general fund"
    # — Core initiative row —————
    initiative_rows.append((
        pg_id,                                # id
        raw_initiative_id,                    # initiative_id
VARCHAR(255)
        entity_type,                          #
initiative_type
        "entities",                          #
source_dynamo_table
        owner_id,                            # owner_id
        e.get("name"),                        # name
        e.get("slug") or None,                # slug
        e.get("status"),                      # status
        e.get("industry") or None,            # industry
        e.get("description") or None,         # description
        _trunc(e.get("color"), 10, "color"),  # color
        e.get("logoUrl") or None,            # logo_url
        e.get("websiteURL") or None,          # website_url
        e.get("cocURL") or None,             # coc_url
        None,                                 # cii_project_id
(project only)
        e.get("stripePlanId") or None,       # stripe_plan_id
        e.get("stripeProductId") or None,    #
stripe_product_id

```

```

        None, #
    jobspring_project_id (project only)
        None, #
    stacks_identififier (project only)
        e.get("eventbriteId") or None, # eventbrite_id
        e.get("applicationURL") or None, #
    application_url
        _as_int(e.get("amountRaised")), # amount_raised
        bool(e.get("acceptFunding", False)), #
    accept_funding
        _parse_ts(e.get("eventStartDate")), #
    event_start_date
        _parse_ts(e.get("eventEndDate")), # event_end_date
        e.get("country") or None, # country
        e.get("city") or None, # city
        bool(e.get("isOnline", False)), # is_online
        _parse_ts(e.get("createdOn")), # created_on
        _parse_ts(e.get("updatedOn")), # updated_on
    ))
# — Entity goals[] —————
# Goal{name, description, goalColor, goalIcon, budget{amount, allocation}}
for idx, goal in enumerate(e.get("goals") or []):
    goal_name = goal.get("name") or f"goal_{idx}"
    budget: dict = goal.get("budget") or {}
    goals_rows.append((
        _uuid5("goal", raw_initiative_id, goal_name),
        raw_initiative_id,
        goal_name,
        _as_int(budget.get("amount")),
        budget.get("allocation") or None,
        None, # repo_link (project only)
        goal.get("description") or None,
        _trunc(goal.get("goalColor"), 10, "goal_color"),
        goal.get("goalIcon") or None,
        idx,
    ))
# — Beneficiaries[] —————
# Entity field: Beneficiary []Beneficiary json:"beneficiaries"
for idx, b in enumerate(e.get("beneficiaries") or []):
    email = b.get("email") or None
    beneficiaries_rows.append((
        _uuid5("beneficiary", raw_initiative_id, email or b.get("name",
        "")) or str(idx)),
        raw_initiative_id, b.get("name") or None, email,
    ))

```

```

# — CustomWebsites[] —————
for cw in (e.get("customWebsites") or []):
    url = cw.get("url") or ""
    if url:
        custom_websites_rows.append((
            _uuid5("custom_website", raw_initiative_id, url),
            raw_initiative_id, cw.get("name") or None, url,
        ))

# — SponsorshipTiers[] —————
# SponsorshipTier{name, description, color, icon, minimum}
for idx, tier in enumerate(e.get("sponsorshipTiers") or []):
    tier_name = tier.get("name") or str(idx)
    sponsorship_tiers_rows.append((
        _uuid5("sponsorship_tier", raw_initiative_id, tier_name),
        raw_initiative_id,
        tier.get("name") or None,
        tier.get("description") or None,
        _trunc(tier.get("color"), 10, "tier_color"),
        tier.get("icon") or None,
        _as_int(tier.get("minimum")),
        idx,
    ))

# — EntityDetails (map[string]string → JSONB) —————
entity_details = e.get("entityDetails")
if entity_details:
    entity_details_rows.append((
        raw_initiative_id,
        _to_jsonb(entity_details),
    ))

# — OSTIF-specific detail (ostif entity type only) —————
# entity.Detail interface{} is a domain.Detail struct when non-nil.
# TypeDeserializer returns it as a plain dict.
if entity_type == "ostif":
    raw_detail = e.get("detail")
    if isinstance(raw_detail, dict) and raw_detail:
        ostif_detail_rows.append((
            raw_initiative_id,
            raw_detail.get("monetizationStrategy") or None,
            raw_detail.get("currentSecurityStrategy") or None,
            raw_detail.get("licenseType") or None,
            _as_int(raw_detail.get("totalBudget")),
            bool(raw_detail.get("termsConditions", False)),
        ))

# Contacts (primaryContact, secondaryContact, technicalLead)
for contact_type, detail_key in (

```

```

        ("primary", "primaryContact"),
        ("secondary", "secondaryContact"),
        ("technical_lead", "technicalLead"),
    ):
        contact = raw_detail.get(detail_key)
        if isinstance(contact, dict) and contact:
            contacts_rows.append((
                _uuid5("contact", raw_initiative_id, contact_type),
                raw_initiative_id,
                contact_type,
                contact.get("firstName") or None,
                contact.get("lastName") or None,
                contact.get("email") or None,
                contact.get("phoneNumber") or None,
                contact.get("otherContactOption") or None,
                contact.get("preferredContactMethod") or None,
            ))

# — Phase 1: Insert initiatives —————
_insert_initiatives(cur, initiative_rows)
# — Phase 2: Insert child tables (FK → initiatives.initiative_id) ———
_insert_goals(cur, goals_rows)
_insert_beneficiaries(cur, beneficiaries_rows)
_insert_custom_websites(cur, custom_websites_rows)
_insert_contributors(cur, contributors_rows)
_insert_mentors(cur, mentors_rows)
_insert_program_info_terms(cur, program_info_terms_rows)
_insert_program_info_skills(cur, program_info_skills_rows)
_insert_program_info_config(cur, program_info_config_rows)
_insert_program_info_custom_term(cur, program_info_custom_term_rows)
_insert_sponsorship_tiers(cur, sponsorship_tiers_rows)
_insert_ostif_detail(cur, ostif_detail_rows)
_insert_contacts(cur, contacts_rows)
_insert_github_stats(cur, github_stats_rows)
_insert_stats(cur, stats_rows)
_insert_entity_details(cur, entity_details_rows)
# — Phase 3: Classify mentorship projects —————
# Projects that have a 'mentee' goal with amount_in_cents > 0 are
# mentorship programs; update initiative_type accordingly.
cur.execute("""
    UPDATE initiatives i
    SET initiative_type = 'mentorship'
    FROM initiative_goals g
    WHERE g.initiative_id = i.initiative_id
        AND g.name = 'mentee'
        AND g.amount_in_cents > 0

```

```

    """
    log.info("    mentorship reclassification : %d row(s) updated", cur.rowcount)
    if skipped:
        log.warning("    → %d initiative(s) skipped (owner not found or missing ID)", skipped)
    log.info("    → %d initiative rows upserted", len(initiative_rows))
    return known_initiative_ids

# -----
# initiative row insert
# -----
def _insert_initiatives(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
        INSERT INTO initiatives (
            id, initiative_id, initiative_type, source_dynamo_table,
            owner_id,
            name, slug, status, industry, description, color,
            logo_url, website_url, coc_url, cii_project_id,
            stripe_plan_id, stripe_product_id,
            jobspring_project_id, stacks_identifier,
            eventbrite_id, application_url,
            amount_raised, accept_funding,
            event_start_date, event_end_date,
            country, city, is_online,
            created_on, updated_on
        ) VALUES (
            %s,%s,%s,%s, %s, %s,%s,%s,%s,%s,%s,
            %s,%s,%s,%s, %s,%s, %s,%s, %s,%s,
            %s,%s, %s,%s, %s,%s,%s, %s,%s
        )
        ON CONFLICT (initiative_id) DO UPDATE SET
            initiative_type      = EXCLUDED.initiative_type,
            source_dynamo_table  = EXCLUDED.source_dynamo_table,
            owner_id             = EXCLUDED.owner_id,
            name                  = EXCLUDED.name,
            slug                  = EXCLUDED.slug,
            status                = EXCLUDED.status,
            industry              = EXCLUDED.industry,
            description           = EXCLUDED.description,
            color                 = EXCLUDED.color,
            logo_url              = EXCLUDED.logo_url,
            website_url           = EXCLUDED.website_url,
            coc_url               = EXCLUDED.coc_url,
            cii_project_id        = EXCLUDED.cii_project_id,

```

```

        stripe_plan_id      = EXCLUDED.stripe_plan_id,
        stripe_product_id   = EXCLUDED.stripe_product_id,
        jobspring_project_id = EXCLUDED.jobspring_project_id,
        stacks_identififier  = EXCLUDED.stacks_identififier,
        eventbrite_id        = EXCLUDED.eventbrite_id,
        application_url      = EXCLUDED.application_url,
        amount_raised        = EXCLUDED.amount_raised,
        accept_funding       = EXCLUDED.accept_funding,
        event_start_date     = EXCLUDED.event_start_date,
        event_end_date       = EXCLUDED.event_end_date,
        country              = EXCLUDED.country,
        city                 = EXCLUDED.city,
        is_online            = EXCLUDED.is_online,
        updated_on           = EXCLUDED.updated_on
    """
    psycopg2.extras.execute_batch(cur, sql, rows, page_size=200)
    log.info("    initiatives          : %d rows", len(rows))
# -----
# Child table inserts
# All use ON CONFLICT (id) DO UPDATE so they are idempotent on re-run.
# UUIDs are generated deterministically via _uuid5 so the same source row
# always produces the same id.
# -----
def _insert_goals(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
        INSERT INTO initiative_goals
            (id, initiative_id, name, amount_in_cents, allocation, repo_link,
             description, color, icon, sort_order)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)
        ON CONFLICT (id) DO UPDATE SET
            amount_in_cents = EXCLUDED.amount_in_cents,
            allocation      = EXCLUDED.allocation,
            repo_link       = EXCLUDED.repo_link,
            description     = EXCLUDED.description,
            color           = EXCLUDED.color,
            icon            = EXCLUDED.icon,
            sort_order      = EXCLUDED.sort_order
    """
    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_goals    : %d rows", len(rows))
def _insert_beneficiaries(cur, rows: list) -> None:
    if not rows:
        return

```

```

sql = """
    INSERT INTO initiative_beneficiaries (id, initiative_id, name, email)
    VALUES (%s,%s,%s,%s)
    ON CONFLICT (id) DO UPDATE SET name = EXCLUDED.name, email =
EXCLUDED.email
"""

psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
log.info("    initiative_beneficiaries: %d rows", len(rows))
def _insert_custom_websites(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
        INSERT INTO initiative_custom_websites (id, initiative_id, name, url)
        VALUES (%s,%s,%s,%s)
        ON CONFLICT (id) DO UPDATE SET name = EXCLUDED.name, url = EXCLUDED.url
    """

    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_custom_websites: %d rows", len(rows))
def _insert_contributors(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
        INSERT INTO initiative_contributors (id, initiative_id, name, email)
        VALUES (%s,%s,%s,%s)
        ON CONFLICT (id) DO UPDATE SET name = EXCLUDED.name, email =
EXCLUDED.email
    """

    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_contributors: %d rows", len(rows))
def _insert_mentors(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
        INSERT INTO initiative_mentors
        (id, initiative_id, name, email, avatar_url, introduction)
        VALUES (%s,%s,%s,%s,%s,%s)
        ON CONFLICT (id) DO UPDATE SET
            name          = EXCLUDED.name,
            email         = EXCLUDED.email,
            avatar_url    = EXCLUDED.avatar_url,
            introduction   = EXCLUDED.introduction
    """

    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_mentors : %d rows", len(rows))
def _insert_program_info_terms(cur, rows: list) -> None:

```

```

    if not rows:
        return
    sql = """
        INSERT INTO initiative_program_info_terms (id, initiative_id, term,
sort_order)
        VALUES (%s,%s,%s,%s)
        ON CONFLICT (id) DO UPDATE SET term = EXCLUDED.term, sort_order =
EXCLUDED.sort_order
    """

    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_program_info_terms: %d rows", len(rows))
def _insert_program_info_skills(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
        INSERT INTO initiative_program_info_skills (id, initiative_id, skill)
        VALUES (%s,%s,%s)
        ON CONFLICT (id) DO UPDATE SET skill = EXCLUDED.skill
    """

    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_program_info_skills: %d rows", len(rows))
def _insert_program_info_config(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
        INSERT INTO initiative_program_info_config (initiative_id,
terms_conditions)
        VALUES (%s,%s)
        ON CONFLICT (initiative_id) DO UPDATE SET
            terms_conditions = EXCLUDED.terms_conditions
    """

    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_program_info_config: %d rows", len(rows))
def _insert_program_info_custom_term(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
        INSERT INTO initiative_program_info_custom_term
            (initiative_id, term_name, start_month, end_month, year)
        VALUES (%s,%s,%s,%s,%s)
        ON CONFLICT (initiative_id) DO UPDATE SET
            term_name = EXCLUDED.term_name,
            start_month = EXCLUDED.start_month,
            end_month = EXCLUDED.end_month,
            year = EXCLUDED.year
    """

```

```

"""
psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
log.info("    initiative_program_info_custom_term: %d rows", len(rows))
def _insert_sponsorship_tiers(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
        INSERT INTO initiative_sponsorship_tiers
            (id, initiative_id, name, description, color, icon, minimum,
sort_order)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
        ON CONFLICT (id) DO UPDATE SET
            name          = EXCLUDED.name,
            description    = EXCLUDED.description,
            color          = EXCLUDED.color,
            icon           = EXCLUDED.icon,
            minimum        = EXCLUDED.minimum,
            sort_order     = EXCLUDED.sort_order
    """

    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_sponsorship_tiers: %d rows", len(rows))
def _insert_ostif_detail(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
        INSERT INTO initiative_ostif_detail
            (initiative_id, monetization_strategy, current_security_strategy,
            license_type, total_budget, terms_conditions)
        VALUES (%s,%s,%s,%s,%s,%s)
        ON CONFLICT (initiative_id) DO UPDATE SET
            monetization_strategy      = EXCLUDED.monetization_strategy,
            current_security_strategy  = EXCLUDED.current_security_strategy,
            license_type                = EXCLUDED.license_type,
            total_budget                = EXCLUDED.total_budget,
            terms_conditions             = EXCLUDED.terms_conditions
    """

    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_ostif_detail: %d rows", len(rows))
def _insert_contacts(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
        INSERT INTO initiative_contacts
            (id, initiative_id, contact_type, first_name, last_name, email,
            phone_number, other_contact_option, preferred_contact_method)
    """

```

```

VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)
ON CONFLICT (id) DO UPDATE SET
    first_name          = EXCLUDED.first_name,
    last_name           = EXCLUDED.last_name,
    email               = EXCLUDED.email,
    phone_number        = EXCLUDED.phone_number,
    other_contact_option = EXCLUDED.other_contact_option,
    preferred_contact_method = EXCLUDED.preferred_contact_method
"""

psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
log.info("    initiative_contacts: %d rows", len(rows))
def _insert_github_stats(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
INSERT INTO initiative_github_stats
    (initiative_id, forks, stars, open_issues)
VALUES (%s,%s,%s,%s)
ON CONFLICT (initiative_id) DO UPDATE SET
    forks          = EXCLUDED.forks,
    stars          = EXCLUDED.stars,
    open_issues    = EXCLUDED.open_issues,
    updated_at     = NOW()
"""
    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_github_stats: %d rows", len(rows))
def _insert_stats(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
INSERT INTO initiative_stats (initiative_id, backers)
VALUES (%s,%s)
ON CONFLICT (initiative_id) DO UPDATE SET
    backers      = EXCLUDED.backers,
    updated_at    = NOW()
"""
    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_stats : %d rows", len(rows))
def _insert_entity_details(cur, rows: list) -> None:
    if not rows:
        return
    sql = """
INSERT INTO initiative_entity_details (initiative_id, details)
VALUES (%s,%s::jsonb)
ON CONFLICT (initiative_id) DO UPDATE SET details = EXCLUDED.details

```

```

"""
    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    log.info("    initiative_entity_details: %d rows", len(rows))
# -----
# Migration: donations (merged from lff-prod-donations +
# lff-prod-entity-donations)
# -----
def migrate_donations(
    cur,
    proj_donations: list,
    entity_donations: list,
    known_users: set,
    known_initiative_ids: set,
    known_org_ids: set,
):
    """
    Merge lff-prod-donations (initiative_id = projectId) and
    lff-prod-entity-donations (initiative_id = entityId) into donations.
    donations.initiative_id is a HARD FK → rows with unknown initiative IDs
    are skipped.
    donations.organization_id maps from the DynamoDB `orgId` attribute and is a
    HARD FK → resolved only when the UUID is in known_org_ids, else NULL.
    """
    log.info(
        "Migrating donations (%d project + %d entity = %d total) ...",
        len(proj_donations), len(entity_donations),
        len(proj_donations) + len(entity_donations),
    )
    sql = """
        INSERT INTO donations (
            id, user_id, initiative_id, organization_id, cached_details, category,
            created_on, current_amount_in_cents, payment_method, status,
stripe_charge_id
        ) VALUES (
            %s,%s,%s,%s,%s::jsonb,%s,%s,%s,%s,%s,%s
        )
        ON CONFLICT (id) DO UPDATE SET
            user_id            = EXCLUDED.user_id,
            initiative_id       = EXCLUDED.initiative_id,
            organization_id     = EXCLUDED.organization_id,
            cached_details      = EXCLUDED.cached_details,
            category            = EXCLUDED.category,
            created_on          = EXCLUDED.created_on,
            current_amount_in_cents = EXCLUDED.current_amount_in_cents,
            payment_method      = EXCLUDED.payment_method,
    """

```

```

        status = EXCLUDED.status,
        stripe_charge_id = EXCLUDED.stripe_charge_id
    """
    rows: list = []
    skipped_user = 0
    skipped_initiative = 0
    # --- lff-prod-donations: PK = (userId, projectId) -----
    for d in proj_donations:
        user_id = d.get("userId")
        if user_id not in known_users:
            skipped_user += 1
            continue
        initiative_id = _as_uuid(d.get("projectId"))
        if initiative_id not in known_initiative_ids:
            skipped_initiative += 1
            log.debug("Skip project donation - initiative %s not found",
initiative_id)
            continue
        pg_id = _uuid5("proj_donation", str(user_id), str(d.get("projectId")))
        org_id = _as_uuid(d.get("orgId"))
        if org_id not in known_org_ids:
            org_id = None # orgId absent or not a known
org
        rows.append((
            pg_id,
            user_id,
            initiative_id,
            org_id,
            _to_jsonb(d.get("cachedDetails")),
            d.get("category"),
            _parse_ts(d.get("createdOn")),
            _as_int(d.get("currentAmountInCents")),
            d.get("paymentmethod"),
            d.get("status"),
            d.get("stripeChargeId"),
        ))
    # --- lff-prod-entity-donations: PK = (userId, entityId) -----
    for d in entity_donations:
        user_id = d.get("userId")
        if user_id not in known_users:
            skipped_user += 1
            continue
        initiative_id = _as_uuid(d.get("entityId"))
        if initiative_id not in known_initiative_ids:
            skipped_initiative += 1

```

```

        log.debug("Skip entity donation - initiative %s not found",
initiative_id)
        continue
    pg_id = _uuid5("entity_donation", str(user_id), str(d.get("entityId")))
    org_id = _as_uuid(d.get("orgId"))
    if org_id not in known_org_ids:
        org_id = None                    # orgId absent or not a known
org
    rows.append((
        pg_id,
        user_id,
        initiative_id,
        org_id,
        _to_jsonb(d.get("cachedDetails")),
        d.get("category"),
        _parse_ts(d.get("createdOn")),
        _as_int(d.get("currentAmountInCents")),
        d.get("paymentmethod"),
        d.get("status"),
        d.get("stripeChargeId"),
    ))
    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    if skipped_user:
        log.warning(" → %d donation(s) skipped (user not found)", skipped_user)
    if skipped_initiative:
        log.warning(" → %d donation(s) skipped (initiative FK not found)",
skipped_initiative)
    log.info(" → %d donation rows upserted", len(rows))
# -----
# Migration: subscriptions (merged from lff-prod-subscriptions +
lff-prod-entity-subscriptions)
# -----
def migrate_subscriptions(
    cur,
    proj_subs: list,
    entity_subs: list,
    known_users: set,
    known_initiative_ids: set,
    known_org_ids: set,
):
    """
    Merge lff-prod-subscriptions (initiative_id = projectId) and
    lff-prod-entity-subscriptions (initiative_id = entityId) into subscriptions.
    subscriptions.initiative_id is a HARD FK → rows with unknown initiative IDs
    are skipped.

```

```

subscriptions.organization_id maps from the DynamoDB `orgId` attribute and
is a HARD FK → resolved only when the UUID is in known_org_ids, else NULL.
"""
log.info(
    "Migrating subscriptions (%d project + %d entity = %d total) ...",
    len(proj_subs), len(entity_subs),
    len(proj_subs) + len(entity_subs),
)
sql = """
INSERT INTO subscriptions (
    id, user_id, initiative_id, organization_id, cached_details, category,
    created_on, current_amount_in_cents, frequency, status,
    stripe_subscription_id, stripe_subscription_item_id
) VALUES (
    %s,%s,%s,%s,%s::jsonb,%s,%s,%s,%s,%s,%s,%s
)
ON CONFLICT (id) DO UPDATE SET
    user_id                = EXCLUDED.user_id,
    initiative_id           = EXCLUDED.initiative_id,
    organization_id         = EXCLUDED.organization_id,
    cached_details         = EXCLUDED.cached_details,
    category                = EXCLUDED.category,
    created_on              = EXCLUDED.created_on,
    current_amount_in_cents = EXCLUDED.current_amount_in_cents,
    frequency               = EXCLUDED.frequency,
    status                  = EXCLUDED.status,
    stripe_subscription_id   = EXCLUDED.stripe_subscription_id,
    stripe_subscription_item_id = EXCLUDED.stripe_subscription_item_id
"""
rows: list = []
skipped_user = 0
skipped_initiative = 0
# --- lff-prod-subscriptions: PK = (userId, projectId) -----
for s in proj_subs:
    user_id = s.get("userId")
    if user_id not in known_users:
        skipped_user += 1
        continue
    initiative_id = _as_uuid(s.get("projectId"))
    if initiative_id not in known_initiative_ids:
        skipped_initiative += 1
        log.debug("Skip project subscription - initiative %s not found",
            initiative_id)
        continue
    pg_id = _uuid5("proj_subscription", str(user_id), str(s.get("projectId")))

```

```

org_id = _as_uuid(s.get("orgId"))
if org_id not in known_org_ids:
    org_id = None # orgId absent or not a known
org
rows.append((
    pg_id,
    user_id,
    initiative_id,
    org_id,
    _to_jsonb(s.get("cachedDetails")),
    s.get("category"),
    _parse_ts(s.get("createdOn")),
    _as_int(s.get("currentAmountInCents")),
    s.get("frequency"),
    s.get("status"),
    s.get("stripeSubscriptionId"),
    s.get("stripeSubscriptionItemId"),
))
# --- lff-prod-entity-subscriptions: PK = (userId, entityId) -----
for s in entity_subs:
    user_id = s.get("userId")
    if user_id not in known_users:
        skipped_user += 1
        continue
    initiative_id = _as_uuid(s.get("entityId"))
    if initiative_id not in known_initiative_ids:
        skipped_initiative += 1
        log.debug("Skip entity subscription - initiative %s not found",
initiative_id)
        continue
    pg_id = _uuid5("entity_subscription", str(user_id),
str(s.get("entityId")))
    org_id = _as_uuid(s.get("orgId"))
    if org_id not in known_org_ids:
        org_id = None # orgId absent or not a known
org
rows.append((
    pg_id,
    user_id,
    initiative_id,
    org_id,
    _to_jsonb(s.get("cachedDetails")),
    s.get("category"),
    _parse_ts(s.get("createdOn")),
    _as_int(s.get("currentAmountInCents")),

```

```

        s.get("frequency"),
        s.get("status"),
        s.get("stripeSubscriptionId"),
        s.get("stripeSubscriptionItemId"),
    ))
    psycopg2.extras.execute_batch(cur, sql, rows, page_size=500)
    if skipped_user:
        log.warning(" → %d subscription(s) skipped (user not found)",
skipped_user)
        if skipped_initiative:
            log.warning(" → %d subscription(s) skipped (initiative FK not found)",
skipped_initiative)
            log.info(" → %d subscription rows upserted", len(rows))
# -----
# Main
# -----
def main():
    log.info("=" * 60)
    log.info("DynamoDB → PostgreSQL migration starting")
    log.info("Region : %s", REGION)
    log.info("PG DSN : %s", PG_DSN)
    log.info("=" * 60)
    dynamo = boto3.client("dynamodb", region_name=REGION)
    # — 1. Scan all DynamoDB tables -----
    users = scan_table(dynamo, "lff-prod-users")
    orgs = scan_table(dynamo, "lff-prod-organizations")
    entities = scan_table(dynamo, "lff-prod-entities")
    projects = scan_table(dynamo, "lff-prod-projects")
    proj_donations = scan_table(dynamo, "lff-prod-donations")
    entity_donations = scan_table(dynamo, "lff-prod-entity-donations")
    proj_subs = scan_table(dynamo, "lff-prod-subscriptions")
    entity_subs = scan_table(dynamo, "lff-prod-entity-subscriptions")
    # — 2. Collect all user IDs referenced in other tables -----
    # Insert placeholder user rows for any referenced ID absent from
lff-prod-users
    # so FK constraints on dependent tables are never violated.
    extra_user_ids: set = set()
    for o in orgs:
        extra_user_ids.add(o.get("ownerId"))
    for e in entities:
        extra_user_ids.add(e.get("ownerId"))
    for p in projects:
        extra_user_ids.add(p.get("ownerId"))
    for d in proj_donations + entity_donations:
        extra_user_ids.add(d.get("userId"))

```

```

for s in proj_subs + entity_subs:
    extra_user_ids.add(s.get("userId"))
known_in_dynamo = {u.get("id") for u in users if u.get("id")}
extra_user_ids -= known_in_dynamo
extra_user_ids.discard(None)
if extra_user_ids:
    log.info(
        "%d user ID(s) referenced in other tables but absent from
lff-prod-users → placeholders",
        len(extra_user_ids),
    )

# — 3. Migrate in FK dependency order —————
log.info("Connecting to PostgreSQL ...")
conn = psycopg2.connect(PG_DSN)
conn.autocommit = False
try:
    with conn.cursor() as cur:
        # 1. user – no FK dependencies
        known_users = migrate_users(cur, users, extra_user_ids)
        conn.commit()
        # 2. organization – FK → user
        known_org_ids = migrate_organizations(cur, orgs, known_users)
        conn.commit()
        # 3. initiatives – FK → user (merged entities + projects)
        known_initiative_ids = migrate_initiatives(cur, entities, projects,
known_users)
        conn.commit()
        # 4. donations – FK → user, FK → initiatives, FK → organizations
(orgId)
        migrate_donations(cur, proj_donations, entity_donations, known_users,
known_initiative_ids, known_org_ids)
        conn.commit()
        # 5. subscriptions – FK → user, FK → initiatives, FK → organizations
(orgId)
        migrate_subscriptions(cur, proj_subs, entity_subs, known_users,
known_initiative_ids, known_org_ids)
        conn.commit()
        log.info("=" * 60)
        log.info("Migration completed successfully!")
        log.info("=" * 60)
    except Exception:
        conn.rollback()
        log.exception("Migration failed – transaction rolled back")
        sys.exit(1)
finally:

```

```

        conn.close()
if __name__ == "__main__":
    main()

```

Data Dictionary

SQL

LFF PostgreSQL Data Dictionary

Version: **2.0.0** | Schema: `db/schema.sql` | Migration:
`db/migrate\_dynamo\_to\_postgres.py`

Conventions

| Symbol | Meaning |

|-----|-----|

| PK | Primary key |

| FK | Foreign key |

| NN | **NOT NULL** |

| UQ | **UNIQUE** constraint |

| `projects.\*` | Source: DynamoDB table `lff-prod-projects` |

| `entities.\*` | Source: DynamoDB table `lff-prod-entities` |

| `donations.\*` | Source: DynamoDB table `lff-prod-donations` |

| `entity-donations.\*` | Source: DynamoDB table `lff-prod-entity-donations` |

| `subscriptions.\*` | Source: DynamoDB table `lff-prod-subscriptions` |

| `entity-subscriptions.\*` | Source: DynamoDB table

`lff-prod-entity-subscriptions` |

| `users.\*` | Source: DynamoDB table `lff-prod-users` |

| `organizations.\*` | Source: DynamoDB table `lff-prod-organizations` |
| **derived** | Value computed at runtime; not persisted in DynamoDB |
| **migration** | Populated only during migration; dropped post-cutover |
Go [JSON](#) tags are noted where they differ from the field name.

Table: `users`

Source: `lff-prod-users`

Go type: `projects/domain.User` · `users/domain.User`

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|--|--------------|-------------|----------------|---|---|
| ----- ----- ----- ----- ----- | | | | | |
| `user_id` | VARCHAR(255) | PK, NN | `id` | `User.ID` / `json:"id"` | Auth0 subject identifier (e.g. `auth0 abc123`). Natural key – not a UUID. |
| `email` | TEXT | | `email` | `User.Email` / `json:"email"` | |
| `given_name` | TEXT | | `givenName` | `User.GivenName` / `json:"givenname"` | |
| Note: Go JSON tag is `givenname` (no camelCase). | | | | | |
| `family_name` | TEXT | | `familyName` | `User.FamilyName` / `json:"familyname"` | |
| `name` | TEXT | | `name` | `User.Name` / `json:"name"` | Full display name. |
| `avatar_url` | TEXT | | `avatarUrl` | `User.AvatarURL` / `json:"avatarUrl"` | |
| `created_on` | TIMESTAMPTZ | | | | DEFAULT NOW() – – Set by DB default; DynamoDB users table has no createdOn. |
| `updated_on` | TIMESTAMPTZ | | | | DEFAULT NOW() – – Set by DB default. |

****Placeholder rows**:** 58 synthetic rows are inserted for user IDs referenced by other tables (donations, subscriptions, initiatives) but absent from `lff-prod-users`. All fields except `user\_id` are [NULL](#).

Table: `organizations`

Source: `lff-prod-organizations`

Go type: `organizations/domain.Organization`

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|-------------------------------|--------------|--------------------------|------------------|-------------------------------|--|
| ----- ----- ----- ----- ----- | | | | | |
| `id` | UUID | PK | `organizationId` | `Organization.OrganizationID` | Deterministically derived from `organizationId` via `_as_uuid()`. |
| `organization_id` | UUID | | `organizationId` | `Organization.OrganizationID` | Duplicate of `id` for backward compatibility. |
| `owner_id` | VARCHAR(255) | NN, FK → `users.user_id` | | `ownerId` | |
| `name` | TEXT | NN | | `Organization.Name` | |
| `avatar_url` | TEXT | | | `Organization.AvatarURL` | |
| `status` | VARCHAR(50) | | | `Organization.Status` | Known values: `active`, `inactive`. |
| `created_on` | TIMESTAMPTZ | | | | DEFAULT NOW() – – Not persisted in DynamoDB organizations. |
| `updated_on` | TIMESTAMPTZ | | | | DEFAULT NOW() – – Not persisted in DynamoDB organizations. |

Table: `initiatives`

Source: `lff-prod-projects` + `lff-prod-entities` (merged)

Go types: `projects/domain.Project` · `entities/domain.Entity`

This table is the central record for every crowdfunding initiative regardless of type. Fields are marked by which DynamoDB source populated them.

Identity columns

| Column | Type | Constraints | DynamoDB field | Source | Notes |
|-----------------------|--------------|-------------|--------------------------|--------------------------|--|
| `id` | UUID | PK | – | both | Surrogate PK. Deterministically generated via `uuid5(projectId\ entityId)` – stable across re-runs. Referenced by `donations.initiative_id` and `subscriptions.initiative_id`. |
| `initiative_id` | VARCHAR(255) | NN, UQ | `projectId` / `entityId` | both | The natural DynamoDB key. VARCHAR(255) not UUID because it is a shared natural key. |
| `initiative_type` | VARCHAR(50) | NN | `entityType` / `project` | both | For projects: hardcoded `project` initially, then reclassified to `mentorship` in Phase 3 if a mentee goal with `amount_in_cents > 0` exists. For entities: taken from `entityType`; DynamoDB quirk reverses `initiative` → `general fund`. Known values: `project`, `mentorship`, `general fund`, `event`, `ostif`. |
| `source_dynamo_table` | VARCHAR(50) | – | *migration* | `projects` or `entities` | **Migration-only column** – drop after cutover (`ALTER TABLE initiatives DROP COLUMN source_dynamo_table`). |

Ownership

| Column | Type | Constraints | DynamoDB field | Source | Notes |
|------------|--------------|-------------|-------------------|-----------|---|
| `owner_id` | VARCHAR(255) | NN, FK | → `users.user_id` | `ownerId` | both Auth0 subject of the project/entity owner. |

Core display fields

| Column | Type | Constraints | DynamoDB field | Go field | Source | Notes |
|---------------|-------------|-------------|----------------|--|--------|---|
| `name` | TEXT | NN | `name` | `Project.Name` / `Entity.Name` | both | Display name. |
| `slug` | TEXT | | `slug` | `Project.Slug` / `Entity.Slug` | both | URL-safe identifier (e.g. `my-project`). |
| `status` | VARCHAR(50) | | `status` | `Project.Status` / `Entity.Status` | both | Projects: `submitted`, `published`, `declined`, `hidden`. Entities: `pending`, `published`. |
| `industry` | TEXT | | `industry` | `Project.Industry` / `Entity.Industry` | both | Free-form industry tag. |
| `description` | TEXT | | `description` | `Project.Description` / `Entity.Description` | both | |
| `color` | VARCHAR(10) | | `color` | `Project.Color` / `Entity.Color` | both | 6-digit hex color code (no `#`). Max 7 chars; truncated at 10 for safety. |
| `logo_url` | TEXT | | `logoUrl` | `Project.LogoURL` / `Entity.LogoURL` | both | |

```

| `website_url` | TEXT | | `website` / `websiteURL` | `Project.Website` /
`Entity.WebsiteURL` | both | Projects use `"website"` JSON key; entities use
`"websiteURL"`. |
| `coc_url` | TEXT | | `codeOfConduct.link` / `cocURL` |
`Project.CodeOfConduct.Link` / `Entity.CocURL` | both | Code of conduct URL.
Projects nest it under `codeOfConduct.link`; entities store it flat. |
### Financial / platform IDs
Column	Type	Constraints	DynamoDB field	Go field	Source	Notes
`stripe_plan_id`	TEXT		`stripePlanId`	`Project.StripePlanID` /		
`Entity.StripePlanID`	both	Stripe recurring-payment plan ID.				
`stripe_product_id`	TEXT		`stripeProductID`	`Project.StripeProductID` /		
`Entity.StripeProductID`	both	Stripe product ID.				
`amount_raised`	INTEGER	NN, DEFAULT 0	`amountRaised`			
`Project.AmountRaised` / `Entity.AmountRaised`	both	Denormalised total				
donations in cents raised. Updated by backend on each donation.						
`accept_funding`	BOOLEAN	DEFAULT false	`acceptFunding`			
`Entity.AcceptFunding` | entities only | Whether the entity is currently accepting
donations. Projects always accept funding when published. |
### Project-only fields
Column	Type	Constraints	DynamoDB field	Go field	Notes
`cii_project_id`	TEXT		`ciiProjectID`	`Project.CIIProjectID`	CII Best
Practices badge programme ID.					
`jobspring_project_id`	TEXT		`jobspringProjectId`		
`Project.JobspringProjectID`	ID of the linked LFX Mentorship (formerly				
Jobspring) project. **Only non-NULL when `initiative_type` = 'mentorship'** – set					
when `ProgramInfo.Budget.AmountInCents > 0`.					
`stacks_identifier`	TEXT		`stacksIdentifier`	`Project.StacksIdentifier`	
Identifier in the Stacks platform. |
### Entity-only fields
Column	Type	Constraints	DynamoDB field	Go field	Notes
`eventbrite_id`	TEXT		`eventbriteId`	`Entity.EventbriteID`	Eventbrite
event ID (event-type entities only).					
`application_url`	TEXT		`applicationURL`	`Entity.ApplicationURL`	URL
for applicants to apply (event entities).					
`event_start_date`	TIMESTAMPTZ		`eventStartDate`	`Entity.EventStartDate`	
Parsed from string via `_parse_ts`.					
`event_end_date`	TIMESTAMPTZ		`eventEndDate`	`Entity.EventEndDate`	
`country`	VARCHAR(100)		`country`	`EntityLocation.Country`	From
embedded `EntityLocation` struct.					
`city`	VARCHAR(100)		`city`	`EntityLocation.City`	
`is_online`	BOOLEAN	DEFAULT false	`isOnline`	`EntityLocation.IsOnline`	

```

Timestamps

| Column | Type | Constraints | DynamoDB field | Notes |
|--------------|-------------|---------------|----------------|--|
| `created_on` | TIMESTAMPTZ | DEFAULT NOW() | `createdOn` | Parsed from 8 possible timestamp formats including Go `time.String()` with nanoseconds + monotonic suffix. |
| `updated_on` | TIMESTAMPTZ | DEFAULT NOW() | `updatedOn` | Same parsing. |

Table: `initiative\_goals`

Source: `lff-prod-projects` (per-category budgets) + `lff-prod-entities` (entity goals array)

Go types: `projects/domain.Budget` · `entities/domain.Goal`

One row per funding category per initiative. Projects produce up to 8 fixed rows; entities produce one row per element in `entity.Goals[]`.

| Column | Type | Constraints | DynamoDB field | Go field | Source | Notes |
|-------------------|--------------|--------------------------------------|---|------------------------|--------|--|
| `id` | UUID | PK - - both | Deterministic: `uuid5("goal", initiative_id, name)` | | | |
| `initiative_id` | VARCHAR(255) | NN, FK → `initiatives.initiative_id` | | | | |
| `name` | TEXT | NN, UQ(initiative_id, name) | | | | Projects: `development`, `marketing`, `meetups`, `travel`, `bugBounty`, `documentation`, `other`, `mentee`. Entities: free-form name from `Goal.Name`. |
| `amount_in_cents` | BIGINT | NN, DEFAULT 0 | | `budget.amount` | | |
| | | | | | | `Budget.AmountInCents` / `json:"amount"` both **JSON tag is `amount` not `amountInCents`.** Stored as cents integer. |
| `allocation` | TEXT | | | `budget.allocation` | | |
| | | | | | | `json:"allocation"` both Free-form description of how the budget is allocated (e.g. `50%`). |
| `repo_link` | TEXT | | | `development.repoLink` | | |
| | | | | | | `json:"repoLink"` projects only Only populated for the `development` goal row. |
| `description` | TEXT | | | `goals[].description` | | |
| | | | | | | `Goal.Description` entities only NULL for project goals. |
| `color` | VARCHAR(10) | | | `goals[].goalColor` | | |
| | | | | | | `Goal.GoalColor` entities only NULL for project goals. Truncated to 10 chars. |
| `icon` | TEXT | | | `goals[].goalIcon` | | |
| | | | | | | `Goal.GoalIcon` entities only NULL for project goals. |
| `sort_order` | INTEGER | DEFAULT 0 | | | | |
| | | | | | | Projects: 0=development, 1=marketing, 2=meetups, 3=travel, 4=bugBounty, 5=documentation, 6=other, 7=mentee. Entities: array index. |

Table: `initiative\_beneficiaries`

Source: `lff-prod-projects` + `lff-prod-entities`

Go type: `projects/domain.Beneficiary` · `entities/domain.Beneficiary`

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|-----------------|--------------|--------------------------------------|---|---------------------|-------|
| `id` | UUID | PK - - | Deterministic: `uuid5("beneficiary", initiative_id, email\ name)` | | |
| `initiative_id` | VARCHAR(255) | NN, FK → `initiatives.initiative_id` | | | |
| `name` | TEXT | | `beneficiaries[].name` | `Beneficiary.Name` | |
| `email` | TEXT | | `beneficiaries[].email` | `Beneficiary.Email` | |

Table: `initiative\_custom\_websites`

Source: `lff-prod-projects` + `lff-prod-entities`

Go type: `projects/domain.CustomWebsite` · `entities/domain.CustomWebsite`

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|-----------------|--------------|--------------------------------------|--|------------------------------------|-----------------------------|
| `id` | UUID | PK - - | Deterministic: `uuid5("custom_website", initiative_id, url)` | | |
| `initiative_id` | VARCHAR(255) | NN, FK → `initiatives.initiative_id` | | | |
| `name` | TEXT | | `customWebsites[].name` | `CustomWebsite.Name` | Display label for the link. |
| `url` | TEXT | NN | `customWebsites[].url` | `CustomWebsite.URL` / `json:"url"` | |

Table: `initiative\_contributors`

Source: `lff-prod-projects` only

Go type: `projects/domain.Contributor`

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|-----------------|--------------|--------------------------------------|---|---------------------|-------|
| `id` | UUID | PK - - | Deterministic: `uuid5("contributor", initiative_id, email\ name)` | | |
| `initiative_id` | VARCHAR(255) | NN, FK → `initiatives.initiative_id` | | | |
| `name` | TEXT | | `contributors[].name` | `Contributor.Name` | |
| `email` | TEXT | | `contributors[].email` | `Contributor.Email` | |

Table: `initiative\_mentors`

Source: `lff-prod-projects` only – present only when `initiative\_type = 'mentorship'`

Go type: `projects/domain.Mentor`

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|-----------------|--------------|--------------------------------------|--|---|-------|
| `id` | UUID | PK - - | Deterministic: `uuid5("mentor", initiative_id, email\ name)` | | |
| `initiative_id` | VARCHAR(255) | NN, FK → `initiatives.initiative_id` | | | |
| `name` | TEXT | | `mentee.mentor[].name` | `Mentor.Name` | |
| `email` | TEXT | | `mentee.mentor[].email` | `Mentor.Email` | |
| `avatar_url` | TEXT | | `mentee.mentor[].avatarURL` | `Mentor.AvatarURL` / `json:"avatarURL"` | |

```
| `introduction` | TEXT | | `mentee.mentor[].introduction` | `Mentor.Introduction`  
| Free-text bio of the mentor. |
```

Table: `initiative\_program\_info\_terms`

Source: `lff-prod-projects` only – present only when `initiative\_type = 'mentorship'`

Go type: `projects/domain.ProgramInfo.Terms` (`[]string`)

```
Column	Type	Constraints	DynamoDB field	Go field	Notes
`id`	UUID	PK	-	-	Deterministic: `uuid5("mentee_term", initiative_id, array_index)`.
`initiative_id`	VARCHAR(255)	NN, FK → `initiatives.initiative_id`			
`term`	TEXT	NN	`mentee.terms[]`	`ProgramInfo.Terms[i]`	Programme term label (e.g. `"Spring 2024"`).
`sort_order`	INTEGER	DEFAULT 0	-	-	Array index from DynamoDB.
```

Table: `initiative\_program\_info\_skills`

Source: `lff-prod-projects` only – present only when `initiative\_type = 'mentorship'`

Go type: `projects/domain.ProgramInfo.Skills` (`[]string`)

```
Column	Type	Constraints	DynamoDB field	Go field	Notes
`id`	UUID	PK	-	-	Deterministic: `uuid5("mentee_skill", initiative_id, skill)`.
`initiative_id`	VARCHAR(255)	NN, FK → `initiatives.initiative_id`			
`skill`	TEXT	NN, UQ(initiative_id, skill)	`mentee.skills[]`		
`ProgramInfo.Skills[i]` | Skill tag; values drawn from `frontend/skills.json`.
```

Table: `initiative\_program\_info\_config`

Source: `lff-prod-projects` only – one row per mentorship initiative

Go type: `projects/domain.ProgramInfo`

```
Column	Type	Constraints	DynamoDB field	Go field	Notes
`initiative_id`	VARCHAR(255)	PK, FK → `initiatives.initiative_id`			
One-to-one with the parent initiative.					
`terms_conditions`	BOOLEAN	DEFAULT false	`mentee.termsConditions`		
`ProgramInfo.TermsConditions` / `json:"termsConditions"` | Whether the project owner has accepted the mentorship programme T&Cs.
```

Table: `initiative\_program\_info\_custom\_term`

Source: `lff-prod-projects` only – present only when `customTerm.termName` is non-empty

Go type: `projects/domain.CustomTerm`

```
Column	Type	Constraints	DynamoDB field	Go field	Notes
```

```

| `initiative_id` | VARCHAR(255) | PK, FK → `initiatives.initiative_id` | | |
One-to-one. |
| `term_name` | TEXT | | `mentee.customTerm.termName` | `CustomTerm.TermName` /
`json:"termName"` | Human-readable term name. |
| `start_month` | VARCHAR(20) | | `mentee.customTerm.startMonth` |
`CustomTerm.StartMonth` / `json:"startMonth"` | Month name or abbreviation (e.g.
`"January"`). |
| `end_month` | VARCHAR(20) | | `mentee.customTerm.endMonth` |
`CustomTerm.EndMonth` / `json:"endMonth"` | |
| `year` | INTEGER | | `mentee.customTerm.year` | `CustomTerm.Year` /
`json:"year"` | 4-digit year. |

```

Table: `initiative\_sponsorship\_tiers`

Source: `lff-prod-entities` only

Go type: `entities/domain.SponsorshipTier`

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|-------------------------------------|--------------|--------------------------------------|-------------------------------|-------------------------------------|--|
| ----- ----- ----- ----- ----- ----- | | | | | |
| `id` | UUID | PK | - | - | Deterministic: `uuid5("sponsorship_tier", initiative_id, name)`. |
| `initiative_id` | VARCHAR(255) | NN, FK → `initiatives.initiative_id` | | | |
| `name` | TEXT | `sponsorshipTiers[].name` | `SponsorshipTier.Name` | Display name (e.g. `Gold Sponsor`). | |
| `description` | TEXT | `sponsorshipTiers[].description` | `SponsorshipTier.Description` | | |
| `color` | VARCHAR(10) | `sponsorshipTiers[].color` | `SponsorshipTier.Color` | Hex color. Truncated to 10 chars. | |
| `icon` | TEXT | `sponsorshipTiers[].icon` | `SponsorshipTier.Icon` | Icon class or URL. | |
| `minimum` | INTEGER | NN, DEFAULT 0 | `sponsorshipTiers[].minimum` | `SponsorshipTier.Minimum` | Minimum donation amount in cents for this tier. |
| `sort_order` | INTEGER | DEFAULT 0 | - | - | Array index from DynamoDB. |

Table: `initiative\_ostif\_detail`

Source: `lff-prod-entities` only – ostif entity type only (one row per ostif initiative)

Go type: `entities/domain.Detail`

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|-------------------------------------|--------------|--------------------------------------|----------------------------------|----------------------------------|-------------------------------|
| ----- ----- ----- ----- ----- ----- | | | | | |
| `initiative_id` | VARCHAR(255) | PK, FK → `initiatives.initiative_id` | | | One-to-one. |
| `monetization_strategy` | TEXT | `detail.monetizationStrategy` | `Detail.MonetizationStrategy` | `json:"monetizationStrategy"` | How the project is monetised. |
| `current_security_strategy` | TEXT | `detail.currentSecurityStrategy` | `Detail.CurrentSecurityStrategy` | `json:"currentSecurityStrategy"` | |

```
| `license_type` | VARCHAR(100) | | `detail.licenseType` | `Detail.LicenseType` /
`json:"licenseType"` | e.g. `"MIT"`, `"Apache 2.0"`. |
| `total_budget` | BIGINT | DEFAULT 0 | `detail.totalBudget` |
`Detail.TotalBudget` / `json:"totalBudget"` | Total security audit budget in
cents. Used as `FundingStatus.TotalAnnualGoalInCents` for ostif entities. |
| `terms_conditions` | BOOLEAN | DEFAULT false | `detail.termsConditions` |
`Detail.TermsConditions` / `json:"termsConditions"` | |
```

--- ## Table: `initiative\_contacts`

Source: `lff-prod-entities` only – ostif entity type only

Go type: `entities/domain.Contact`

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|---|--------------|-------------|-------------------------------|-----------------------------|---|
| `id` | UUID | PK | – | – | Deterministic: `uuid5("contact", initiative_id, contact_type)`. |
| `initiative_id` | VARCHAR(255) | NN, FK | → | `initiatives.initiative_id` | |
| `contact_type` | VARCHAR(50) | NN, UQ | (initiative_id, contact_type) | – | – |
| `primary`, `secondary`, or `technical_lead`. Derived from which DynamoDB key the contact appears under (`primaryContact`, `secondaryContact`, `technicalLead`). | | | | | |
| `first_name` | TEXT | | `detail.{type}.firstName` | `Contact.FirstName` | / |
| `json:"firstName"` | | | | | |
| `last_name` | TEXT | | `detail.{type}.lastName` | `Contact.LastName` | / |
| `json:"lastName"` | | | | | |
| `email` | TEXT | | `detail.{type}.email` | `Contact.Email` | / `json:"email"` |
| `phone_number` | VARCHAR(50) | | `detail.{type}.phoneNumber` | `Contact.PhoneNumber` | / `json:"phoneNumber"` |
| `Contact.OtherContactOption` / `json:"otherContactOption"` Alternative contact method description. | | | | | |
| `preferred_contact_method` VARCHAR(50) | | | | | |
| `detail.{type}.preferredContactMethod` `Contact.PreferredContactMethod` / | | | | | |
| `json:"preferredContactMethod"` e.g. `"email"`, `"phone"`. | | | | | |

--- ## Table: `initiative\_github\_stats`

Source: `lff-prod-projects` only

Go type: `projects/domain.GithubStats`

Updated independently via `UpdateGithubDataCache` (targeted DynamoDB `UpdateItem` on `cachedDetails.githubStats.\*`).

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|-----------------|--------------|---------------|-----------------------------------|-----------------------------|------------------|
| `initiative_id` | VARCHAR(255) | PK, FK | → | `initiatives.initiative_id` | |
| One-to-one. | | | | | |
| `forks` | INTEGER | NN, DEFAULT 0 | `cachedDetails.githubStats.forks` | `GithubStats.Forks` | / `json:"forks"` |

```
| `stars` | INTEGER | NN, DEFAULT 0 | `cachedDetails.githubStats.stars` |
`GithubStats.Stars` / `json:"stars"` | |
| `open_issues` | INTEGER | NN, DEFAULT 0 | `cachedDetails.githubStats.openIssues`
| `GithubStats.OpenIssues` / `json:"openIssues"` | |
| `updated_at` | TIMESTAMPTZ | DEFAULT NOW() | - | - | Set by DB default on
upsert. |
```

--- ## Table: `initiative\_stats`

Source: `lff-prod-projects` only

Go type: `projects/domain.ProjectStats`

Updated independently via `UpdateProjectStats` (targeted DynamoDB `UpdateItem` on `cachedDetails.projectStats.backers`).

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|-----------------|--------------|--------------------------------------|--------------------------------------|------------------------|---|
| `initiative_id` | VARCHAR(255) | PK, FK → `initiatives.initiative_id` | | | One-to-one. |
| `backers` | INTEGER | NN, DEFAULT 0 | `cachedDetails.projectStats.backers` | `ProjectStats.Backers` | / `json:"backers"` Count of unique donors. Incremented/decremented on each donation/cancellation. |
| `updated_at` | TIMESTAMPTZ | DEFAULT NOW() | | | |

****Excluded field**:** `ProjectStats.TotalRaised` (`json:"totalRaised"`) – this field has no active write path in the backend; `convertProjectToDynamoRepresentation` never sets it. Always 0 in DynamoDB.

--- ## Table: `initiative\_entity\_details`

Source: `lff-prod-entities` only

Go type: `entities/domain.EntityDetails` (`map[string]string`)

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|-----------------|--------------|--------------------------------------|-----------------|------------------------|---|
| `initiative_id` | VARCHAR(255) | PK, FK → `initiatives.initiative_id` | | | One-to-one. |
| `details` | JSONB | NN, DEFAULT `{}` | `entityDetails` | `Entity.EntityDetails` | / `json:"entityDetails"` Arbitrary `map[string]string` of application-defined metadata. Serialised by `dynamodbattribute.MarshalMap`. Keys and values are application-specific and vary by entity type. |

--- ## Table: `donations`

Source: `lff-prod-donations` + `lff-prod-entity-donations` (merged)

Go types: `donations/domain.Donation` · `donations/domain.EntityDonation`

DynamoDB model: `donations/interfaces/repository.dynamoDonation` / `entityDonation`

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|--------|------|-------------|----------------|----------|---|
| `id` | UUID | PK - - | | | Deterministic: `uuid5("donation", user_id, initiative_id, created_on)`. |

| `user\_id` | VARCHAR(255) | NN, FK → `users.user\_id` | `userId` |
`Donation.UserID` | Auth0 subject of the donor. |
| `initiative\_id` | UUID | FK → `initiatives.id` | `projectId` / `entityId` |
`Donation.ProjectID` / `EntityDonation.EntityID` | References the ****surrogate UUID PK**** (`initiatives.id`), not the natural key. May be NULL if initiative was not found. |
| `organization\_id` | UUID | FK → `organizations.id` | `orgId` | `Donation.OrgID`
(`\*string`) | Optional. Set when donation is made on behalf of an organisation. |
| `cached\_details` | JSONB | | `cachedDetails` | `cachedDetails` struct | Snapshot of backer metadata at donation time: `{backerDetails: {name, avatarURL}}`. |
| `category` | TEXT | | `category` | `Donation.Category` (`\*string`) | Funding category the donation is directed to (e.g. `development`, `marketing`, `mentee`). NULL means "all needs". In email rendering, `mentee` is displayed as
`"Mentorship"`. |
| `created\_on` | TIMESTAMPTZ | DEFAULT NOW() | `createdOn` | `Donation.CreatedOn`
| Parsed from RFC3339 string. |
| `current\_amount\_in\_cents` | BIGINT | NN | `currentAmountInCents` |
`Donation.AmountInCents` / `json:"currentAmountInCents"` | Donation amount in US cents (e.g. `500` = \$5.00). |
| `payment\_method` | VARCHAR(50) | | `paymentmethod` | `Donation.PaymentMethod` |
`card` or `invoice`. Note: DynamoDB key is `paymentmethod` (lowercase, no camelCase). |
| `status` | VARCHAR(50) | | `status` | `Donation.Status` (`DonationStatus`) |
Known values: `succeeded`, `failed`, `pending`. |
| `stripe\_charge\_id` | VARCHAR(255) | | `stripeChargeId` |
`Donation.StripeChargeID` / `json:"stripeChargeId"` | Stripe charge object ID (e.g. `ch\_abc123`). |

Table: `subscriptions`

Source: `lff-prod-subscriptions` + `lff-prod-entity-subscriptions` (merged)

Go types: `subscriptions/domain.Subscription` ·

`subscriptions/domain.EntitySubscription`

DynamoDB model: `subscriptions/interfaces/repository.dynamoSubscription` /

`dynamoEntitySubscription`

| Column | Type | Constraints | DynamoDB field | Go field | Notes |
|--------------------------|--------------|--------------------------|---|----------|--|
| `id` | UUID | PK - - | Deterministic: `uuid5("subscription", user_id, initiative_id, created_on)`. | | |
| `user_id` | VARCHAR(255) | NN, FK → `users.user_id` | `userId` | | |
| `Subscription.UserID` | | | | | |
| `initiative_id` | UUID | FK → `initiatives.id` | `projectId` / `entityId` | | |
| `Subscription.ProjectID` | | | `EntitySubscription.EntityID` | | References surrogate UUID PK. May be NULL if initiative not found. |

```

| `organization_id` | UUID | FK → `organizations.id` | `orgId` |
`Subscription.OrgID` (`*string`) | Optional. Organisation on whose behalf the
subscription was created. |
| `cached_details` | JSONB | | `cachedDetails` | `cachedDetails` struct | Snapshot
of backer metadata: `{backerDetails: {name, avatarURL}}`. |
| `category` | TEXT | | `category` | `Subscription.Category` (`*string`) | Funding
category. Same values as `donations.category`. NULL = all needs. |
| `created_on` | TIMESTAMPTZ | DEFAULT NOW() | `createdOn` |
`Subscription.CreatedOn` | Parsed from string. |
| `current_amount_in_cents` | BIGINT | NN | `currentAmountInCents` |
`Subscription.AmountInCents` / `json:"currentAmountInCents"` | Monthly or annual
recurring amount in US cents. |
| `frequency` | VARCHAR(50) | | `frequency` | `Subscription.Frequency`
(`*SubscriptionFrequency`) | `monthly` or `yearly`. Uses
`stripeservice.SubscriptionFrequency` constants. |
| `status` | VARCHAR(50) | | `status` | `Subscription.Status` | `active` or
`inactive`. |
| `stripe_subscription_id` | VARCHAR(255) | | `stripeSubscriptionId` |
`Subscription.StripeSubscriptionID` / `json:"stripeSubscriptionId"` | Stripe
subscription object ID (e.g. `sub_abc123`). |
| `stripe_subscription_item_id` | VARCHAR(255) | | `stripeSubscriptionItemId` |
`Subscription.StripeSubscriptionItemId` / `json:"stripeSubscriptionItemId"` |
Stripe subscription item ID (e.g. `si_abc123`). Needed for per-item updates. |

```

Excluded fields

These fields appear in the Go domain structs and/or DynamoDB but are ****not persisted**** to PostgreSQL because they have no DynamoDB write path or are computed at read time.

| Field | Go type / location | Reason excluded |
|-------------------------|---|---|
| `balance` | `projects/domain.Balance` / `entities/domain.Balance` | Computed at read time by calling the Ledger microservice API (`GetEntityBalanceFromStartDate`). Never written to DynamoDB by `SaveProject` or `SaveEntity`. |
| `funding_status` | `projects/domain.FundingStatus` / `entities/domain.FundingStatus` | Computed from `balance` + subscription summary aggregates. Populated by `fillFundingStatus` at request time. |
| `entity_stats.sponsors` | `entities/domain.EntityStats.Sponsors` | `GetEntitySponsors` pushes to Elasticsearch only. `SaveEntity` is never called with `EntityStats` populated. |
| `sponsors` | `projects/domain.Sponsor[]` / `entities/domain.Sponsor[]` | `GetEntitySponsors` / `GetProjectSponsors` push to ES only. |
| `diversity` | `projects/domain.Diversity` | `GetDiversity` fetches from an external demographic API at read time and does not persist results. |

| `vulnerability\_summary` | `projects/domain.VulnerabilitySummary` |
 `GetVulnerabilitySummary` fetches from an external security scanning service at
 read time. |
 | `badges` | `projects/domain.Badge[]` | `convertProjectToDynamoRepresentation`
 never assigns `Badges`. The field is always nil/empty in DynamoDB. |
 | `project\_stats.total\_raised` | `projects/domain.ProjectStats.TotalRaised` | No
 active write path. `UpdateProjectStats` only increments/decrements `backers`.
 Value is always 0 in DynamoDB. |
 | `entity\_stats.total\_raised` | `entities/domain.EntityStats.TotalFundsRaised` |
 Same as above for entities. |
 | `cii\_markup` | `projects/domain.ProjectDetails.CIIMarkup` | Fetched live from
 `bestpractices.coreinfrastructure.org` at request time, not stored. |
 | `po\_number` | `donations/domain.Donation.PONumber` | Present in DynamoDB
 (`ponumber`) but not mapped in schema – purchase order number used only for
 invoice-payment email flows. |
 | `uncategorised` | `projects/domain.Uncategorised` | "All project needs"
 pseudo-category; not a true budget category and has no independent representation
 in goals. |
 | `project\_funding\_status` | `projects/domain.ProjectFundingStatus` | Per-category
 funding breakdown; computed from ledger + subscriptions at read time. |

Indexes

| Index name | Table | Column(s) | Purpose |
|--------------------------------------|------------------------------|-------------------|--|
| ----- ----- ----- ----- | | | |
| `idx_organizations_owner_id` | `organizations` | `owner_id` | Look up orgs by owner. |
| `idx_initiatives_initiative_id` | `initiations` | `initiative_id` | Fast natural-key lookup (though covered by UNIQUE constraint). |
| `idx_initiatives_owner_id` | `initiations` | `owner_id` | All initiatives owned by a user. |
| `idx_initiatives_slug` | `initiations` | `slug` | URL routing by slug. |
| `idx_initiatives_status` | `initiations` | `status` | Filter published/submitted/declined. |
| `idx_initiatives_type` | `initiations` | `initiative_type` | Filter by type (project/mentorship/event/etc). |
| `idx_initiatives_amount_raised` | `initiations` | `amount_raised` | DESC Discovery sort by funding. |
| `idx_initiative_goals_iid` | `initiative_goals` | `initiative_id` | Load all goals for an initiative. |
| `idx_initiative_beneficiaries_iid` | `initiative_beneficiaries` | `initiative_id` | |
| `idx_initiative_custom_websites_iid` | `initiative_custom_websites` | `initiative_id` | |
| `idx_initiative_contributors_iid` | `initiative_contributors` | `initiative_id` | |

```
| `idx_initiative_mentors_iid` | `initiative_mentors` | `initiative_id` | |
| `idx_initiative_program_info_terms_iid` | `initiative_program_info_terms` |
`initiative_id` | |
| `idx_initiative_program_info_skills_iid` | `initiative_program_info_skills` |
`initiative_id` | |
| `idx_initiative_sponsorship_tiers_iid` | `initiative_sponsorship_tiers` |
`initiative_id` | |
| `idx_initiative_contacts_iid` | `initiative_contacts` | `initiative_id` | |
| `idx_donations_user_id` | `donations` | `user_id` | All donations by a user. |
| `idx_donations_initiative_id` | `donations` | `initiative_id` | All donations to
an initiative. |
| `idx_donations_status` | `donations` | `status` | Filter
succeeded/failed/pending. |
| `idx_donations_org_id` | `donations` | `organization_id` | All donations by an
org. |
| `idx_subscriptions_user_id` | `subscriptions` | `user_id` | All subscriptions by
a user. |
| `idx_subscriptions_initiative_id` | `subscriptions` | `initiative_id` | All
subscriptions to an initiative. |
| `idx_subscriptions_org_id` | `subscriptions` | `organization_id` | |
| `idx_subscriptions_status` | `subscriptions` | `status` | Filter
active/inactive. |
```